The Concept Hierarchy: Syntax Reference V0

A Companion Technical Document

Andrei Costinescu

16.08.2026

Abstract

Applications that reason over a domain typically maintain two artifacts: an ontology, stating what the domain's entities are and how they relate, and a program, stating what is computed from them. The two must manually be always made consistent for applications to operate on the same knowledge. This is done mostly by the discipline and rigorousness of the programmer, which is subject to human errors; nothing checks that the two artifacts are always in agreement.

The Concept Hierarchy is a knowledge programming language that removes the split and presents both artifacts into one framework: an ontology and a type system are defined as one hierarchy of concepts, in one `JSON` document, so that a domain category and the data type of a value it is described by are related by a single subsumption relation and checked against each other by a single compiler. Domain knowledge so expressed is uncertain by construction: every property value carries a confidence, and the language maintains consistency between related properties through modeler-defined procedures.

This document is the syntax reference for the Concept Hierarchy language. It specifies, for every construct, the `JSON` form the construct takes and the conditions under which that form is well-formed: the three kinds of concept (*DomainConcepts*, *ValueDomains*, and *Functions*), the parameterization and instantiation schemas through which *ValueDomains* define *Types*, the reification by which a *DomainConcept* is made available where a *Type* is expected, the expression grammar and the procedure that classifies a `JSON` value against it, the global instance database, and the inclusion of external files. Well-formedness conditions are stated as definitions and are decidable from the text of a Concept Hierarchy alone; the execution semantics of a Concept Hierarchy are not specified here. The document is a standalone reference, complementing the language's design and semantics.

Contents

1 Introduction

1.1 The Concept Hierarchy

A Concept Hierarchy is a single `JSON` document that defines a body of knowledge and the programs that operate on it. Its concepts are arranged in a single subsumption hierarchy rooted at `Concept`, and every concept is one of three kinds. A *DomainConcept* models a category of entities in the application domain and states what can be known about its instances. A *ValueDomain* defines a data type, i.e. a set of values, and may be parameterized over *Types* and literals. A *Function* is a *ValueDomain* that is callable, and is the only construct able to change a value. The two top-level sections of every Concept Hierarchy document that carry knowledge (**`"concepts"`** and **`"instances"`**) separate the definitions from the data: **`"concepts"`** carries the concept definitions, and **`"instances"`** the global variables over which a Concept Hierarchy's expressions are written.

Two features distinguish the resulting language from an ontology language and from a programming language alike. First, the ontology axioms and the type system share the same hierarchy and definition syntax, so a *DomainConcept* and a *ValueDomain* share their structural keywords and are related by a single subsumption relation. Second, every property value carries a confidence in $[0, 1]$ as part of the data model rather than as an

attribute added beside it, so a Concept Hierarchy records no value as simply correct. Section 3 develops both points and provides the definition template that all three kinds of concepts share.

1.2 Purpose and Scope

This document specifies the concrete syntax of a Concept Hierarchy and the static well-formedness conditions that a Concept Hierarchy must satisfy. For every construct, it fixes the `JSON` form the construct takes, the conditions under which that form is admissible, and what the construct denotes so far as that is determined statically. Conditions of the third kind are stated as definitions rather than left to prose, and each is decidable from the text of a Concept Hierarchy alone: no condition stated here requires executing a Concept Hierarchy, inspecting its runtime state, or consulting the host-language implementations that realize its *ValueDomains* and *Functions*.

The semantic behavior of a Concept Hierarchy is correspondingly outside the scope of this document. It comprises the evaluation of a *Function*'s body, the consistency maintaining system between related properties, the composition of confidence across values computed from other values, the registration and revocation of *DomainConcepts* on an instance as its property values change, and the creation of instances during execution. Where a definition keyword governs such behavior, this document states what the keyword's declared value means for one value at one point in time, and no more. The rule is deliberate rather than incidental: what a keyword *declares* is fixed by the text and belongs here, whereas what it *does* depends on a state that the text does not determine.

Throughout the document, example concepts are defined. They are meant to illustrate modeling options and are not to be understood as the best modeling option or even that every Concept Hierarchy definition must contain them. Appendix B presents the concepts that must be defined to enable particular features in a Concept Hierarchy.

1.3 Audience

The document addresses two readers. For an author writing a Concept Hierarchy, it fixes what may be written at each position of the document and what each written form means. For an implementer of a Concept Hierarchy compiler, it fixes the checks the compiler must perform before a Concept Hierarchy is admitted, and, where a condition is deliberately not checked statically, says so and gives the reason. Familiarity with `JSON` and `JSON Schema` is assumed throughout, as is a working acquaintance with generic types in the sense of `C++` templates and with the standard axioms of ontology design. No familiarity with the Concept Hierarchy is assumed.

1.4 Structure of This Document

Section 2 fixes the naming conventions for Concept Hierarchy identifiers and the semantic highlighting used in every subsequent code listing. Section 3 gives the top-level structure of a Concept Hierarchy document and the definition template shared by all three concept kinds, including the structural axioms of ontology design that the template's keywords express.

The four sections that follow define the concept kinds and the mechanism through which the ontology becomes usable as programming language *Types*. Section 4 defines *DomainConcepts*: their **`properties`** and **`functions`**, the mechanism by which either may be specialized as the hierarchy is refined, and the **`management`** procedures that restore consistency after an instance is created or cloned. Section 5 defines *ValueDomains*: their template parameters and the language constraining them, the syntax of a type application, the substitution of a parent's template arguments, and the `JSON Schema` dialect in which a *ValueDomain*'s **`"instantiation"`** is written. Section 6 defines the family of *ValueDomains* through which a *DomainConcept* is reified, i.e. made writable wherever a *Type* is expected, and gives the subtype relation over that family. Section 7 defines *Functions*: their **`"interface"`**, the three operations of evaluation, composition, and instantiation, and the two keywords introducing local variables.

The last three sections concern what a Concept Hierarchy is assembled from. Section 8 defines the expressions occurring at every value position of the preceding sections, the scopes against which a variable is resolved, the procedure classifying a raw `JSON` value into an expression form, and the admissibility of a classified expression at its site. Section 9 defines the **`"instances"`** section and the global variables it binds. Section 10 defines the **`"external"`** keyword, by which one hierarchy is distributed over several files, and the conditions under which those files assemble into one Concept Hierarchy.

The two appendices summarize the keywords used to define Concept Hierarchies (Appendix A) and enumerate which features of the language are enabled by defining *ValueDomains* and *Functions* (Appendix B).

A reader approaching the language for the first time is advised to read section 3 and section 4 in order, then section 8, and to consult section 5 as its definitions are referenced from elsewhere; the type-system material is the most self-contained and the least prerequisite to the rest.

1.5 Accompanying Artifacts

Two machine-readable specifications accompany this document and are maintained with it: an `EBNF` grammar and a `JSON Schema` for the Concept Hierarchy document format.[1] The `EBNF` productions reproduced in this document, in Listing 10, Listing 11, Listing 12, and Listing 16, are excerpts of that grammar, given where the constructs they generate are defined. Neither artifact expresses the well-formedness conditions of this document that range over more than one definition, and neither is a substitute for it. The design of the language, the rationale for its commitments, its formal semantics, and its evaluation are not included.

2 Notational Conventions

Two conventions apply to every listing and every mention of Concept Hierarchy code in this document. The first is lexical, and is a rule of the language rather than of its presentation: Table 1 gives the form each kind of Concept Hierarchy identifier must take, so that the syntactic category of a name is recoverable from the name itself. Capitalization is load-bearing here and not stylistic, since it is what keeps a concept name, a property name, a template parameter, and a reserved keyword apart at positions where the grammar admits more than one of them. The second is presentational: Concept Hierarchy code is set with the semantic highlighting of Table 2, which assigns a color to each syntactic role and, within a role, distinguishes reserved keywords by weight from author-chosen names. The scheme extends to the `JSON Schema` dialect of subsection 5.4, whose custom keywords are distinct from those that carry their Draft-07 meaning. Both tables are reference material and need not be read through; each row is reintroduced when the identifier or the construct it governs is defined.

Table 1: Naming conventions for Concept Hierarchy identifiers. All identifiers are strings containing alphanumeric characters or the underscore character (`_`); `object` key names (*`arguments`*) do not have a convention in a *ValueDomain*'s **`instantiation`** schema; this allows the Concept Hierarchy to interact with existing, external data (possibly having with other conventions) via a *ValueDomain*'s **`instantiation`** schema.

Identifier	**Naming Convention**
Concept names	Uppercase-initial string
Variable (instance) names	Letter-initial string
DomainConcept properties	Lowercase-initial string (unique across the entire Concept Hierarchy, incl. function names)
DomainConcept functions	Lowercase-initial string (unique across the entire Concept Hierarchy, incl. property names)
ValueDomain template parameters	Uppercase-initial string (must be different from concept names)
Function evaluation arguments	Lowercase-initial string

3 Concept Hierarchy Overview

The Concept Hierarchy adopts `JSON` [International(2017)] as its concrete syntax, valuing its structured yet human-readable format over `XML` or over `OWL`'s typical RDF/XML serialization. `OWL` ontologies are conventionally authored through a graphical editor and serialized to RDF/XML, a format not intended for direct human editing [Musen(2015)]; the Concept Hierarchy instead defines both its ontology and its type system directly in `JSON`.

A Concept Hierarchy minimally defines concepts – the data types, functions, and domain data of the language – and may additionally define instances – the global variables available for use in expressions throughout the

[1] Respectively available in GitHub: EBNF and JSONSchema.

Table 2: Semantic highlighting in Concept Hierarchy code

Usage (bold = reserved/fixed keywords, italics markings)	**Color**
Names of *DomainConcepts*, (instantiated) *ValueDomains*, and (instantiated) *Functions*	
Template parameters of ValueDomains and Functions	
Concept definition keywords, incl. *DomainConcept*, *ValueDomain*, and *Function* keywords	
Property and function names of *DomainConcepts*	
Property and function keywords in *DomainConcepts*	
Keywords to define specialization scope of *DomainConcepts* properties and functions	
Keywords to define pre and post hooks for *DomainConcepts* properties	
ValueDomain instantiation and Function evaluation arguments	
For global, local, and implicit variables (the latter are, e.g., concept properties in a `hook` function)	
Expressions (e.g. *ValueDomain* instantiations or *Function* evaluations)	
Whether the *Function* specification is a procedure evaluation or a composition definition	
Provenance character of ValueDomain instantiation and Function evaluation arguments	
Access character of *Function* evaluation arguments	
Comments (explanatory and placeholders)	
For characters `"[]{}.,:` in `JSON` syntax and non-Concept Hierarchy expressions	
Metakeywords, e.g. including external files into the Concept Hierarchy	
Concept Hierarchy code that has errors and is invalid	
`JSON Schema` custom keywords: type, provenance, default, properties, format, constraint	
`JSON Schema` keywords with Draft-07 meaning (type, properties, items, pattern, ...)	
`JSON Schema` keywords referencing data ($ref, $defs, ...) & values of $ref, and $defs entries	
`JSON Schema` user-defined values for Draft-07 keywords (e.g. in patternProperties)	

Concept Hierarchy's concept and instance definitions – and merge in content from external files. Concepts are defined in a Concept Hierarchy's **`"concepts"`** keyword, global variables in **`"instances"`**, and external content in **`"external"`**. Optionally, the defined Concept Hierarchy may be named under its top-level **`"name"`** keyword, and metadata, such as the version of the model or compiler instructions for example, may be specified in the **`"metadata"`** keyword of a Concept Hierarchy definition.

In the Concept Hierarchy, every concept type – data types, functions, and domain data alike – is represented as a subconcept of `Concept`, the (possibly implicit) root of the hierarchy. Listing 1 gives the general template shared by every concept definition; its keywords are structural (fixing a concept's place in the hierarchy), cardinality-related (bounding its population), or purely documentary, with the mandatory/optional status of each detailed in the caption.

```
{
  |...|,
  |string: uppercase-starting concept name|: {
    "description": |string: textual concept description for documentation|,
    "data": |object: concept-type specific data definition|,
    "directParents": |array of strings: list of direct parent concept names (SUBSUMPTION)|,
    "directChildren": |array of strings: full list of direct children concept names (COVERING)|,
    "distinctFrom": |array of strings: list of names of concepts whose instances can never be
    instances of this concept (DISJOINTNESS)|,
    "distinctGroup": |array of strings: list of direct children concept names that are disjoint (
    DISJOINTNESS; syntactic-sugar: is converted to "distinctFrom" in children concepts)|,
    "minInstances": |int: minimal number of instances so that the Concept Hierarchy is valid|,
    "maxInstances": |int: maximal number of instances so that the Concept Hierarchy is valid|,
    "abstract": |boolean: whether direct instances of this concept (excluding subconcepts) can exist
    |
  },
  |string: name that aliases (EQUIVALENCE)|: |string: aliased concept or type application|
}
```

Listing 1: General template for a concept definition in the Concept Hierarchy. **`"directParents"`** is mandatory for

non-root concepts, and, if there are one or more concepts that do not specify **"directParents"** and Concept is not defined in the hierarchy, these concepts become the direct children of Concept. All remaining keywords are optional: **"data"**, **"description"**, **"directChildren"**, **"distinctFrom"**, **"distinctGroup"**, **"minInstances"** (default 0), **"maxInstances"** (default ∞), and **"abstract"** (default false).

These structural keywords give the Concept Hierarchy direct syntactic counterparts to the standard set-theoretic axioms of ontology design: concept aliasing expresses *equivalence*, **"directParents"** expresses *subsumption*, **"distinctFrom"** and **"distinctGroup"** express *disjointness*[2], and **"directChildren"** expresses *covering/exhaustiveness*. Two standard axioms remain unsupported when defining a concept: *complementation* (that only the instances outside a concept populate its complement) and *enumeration* (explicitly listing a concept's instances). Complementation can be achieved through reification (subsection 6.3), but not as an ontological axion.

The remaining keywords govern how a concept may be instantiated and must be satisfied at all times during a Concept Hierarchy's lifetime. **"minInstances"** and **"maxInstances"** bound the population of a concept – and, transitively, of all its subconcepts – at every point in a valid Concept Hierarchy. **"abstract"**, by contrast, forbids only *direct* instantiation of the concept itself; subconcepts are unaffected and must independently declare **"abstract"** if they, too, are to be non-instantiable. The two keywords are therefore not interchangeable: **"abstract"**: true restricts only direct instances of a concept, whereas **"maxInstances"**: 0 restricts instances of the concept and of all its transitive subconcepts alike.

Concept aliasing is a method of adding a different name to the same concept. Either name may be used throughout the text of a Concept Hierarchy. An alias may reference another alias, but every alias chain must end at a concept or *Type* application of the Concept Hierarchy. A *Type* application can also be an alias, enabling the modeler, for example, to define "Position": "Vector<3>", making Position a (type) alias of Vector<3>. Aliases are governed by the same rules as the names they alias. For example, a *Type*-application alias may not be specified as a **directParents** entry.

Building on this common concept-definition-template, the Concept Hierarchy distinguishes three concept types: *ValueDomains* and *Functions*, which together provide the (classical) programming-language features of a Concept Hierarchy, and *DomainConcepts*, which carry the property-specific ontology and description-logic (DL) constraints of the domain.

DomainConcepts are all concepts that are not subconcepts of *ValueDomain*. Thus, Concept is a *DomainConcept*, but ValueDomain is not. Thus, there is a disjoint partition of concepts between *DomainConcepts* and *ValueDomains*. Furthermore, the set of *Function* concepts is a subset of the set of *ValueDomains* and it is defined as all the concepts that are subconcepts of Function, which must be a (direct) subconcept of ValueDomain. We detail the **"data"** content specific to each concept type in the subsections that follow.

4 Definition of *DomainConcepts*

A *DomainConcept* models a category of entities in the application domain: it names a class of individuals and specifies what can be known about them. Collectively, the *DomainConcepts* of a Concept Hierarchy constitute its *ontology* — an explicit, machine-readable account of the domain's structure. Representing that structure explicitly, rather than leaving it implicit in the logic of the programs that consume the data, is what allows the structure to be shared across programs, reused across applications, and inspected independently of any one of them; it also turns the modeling assumptions about the domain into a first-class, inspectable artifact rather than something buried in code [Noy and McGuinness(2001)].

An ontology is easily mistaken for an object model, since both group individuals into classes arranged in an inheritance hierarchy, each class carrying named attributes. The two are distinguished less by their form than by what drives the modeling. Object-oriented class design is organized around a class's *operational* role — the operations that will run on it — whereas ontological modeling is organized around the *structural* properties of the modeled entities themselves, independently of any program that manipulates them [Noy and McGuinness(2001)]. A *DomainConcept* is deliberately placed on both sides of this line: it gives a *structural* description of a domain category through its **properties**, and attaches *behavior* to that category through its **functions**, with both organized under the concept's single subsumption hierarchy.

[2]The sets that the two concepts denote share no member, and, hence, no concept may descend from both.

Concretely, a *DomainConcept*'s **data** is organized into the three keys shown in Listing 2: **properties**, the concept's structural description; **functions**, behavior attached to its instances; and **management**, the procedures that restore consistency after an instance is created or cloned. The remainder of this subsection defines each in turn, together with the mechanism by which properties and functions may be *specialized* as the hierarchy is refined into more specific concepts.

```
{
  |...|,
  "DomainConceptExample": {
    "directParents": ["Concept"],
    "data": {
      "properties": { |property names -> property definition data| },
      "functions": { |function names -> function definition data| },
      "management": {
        "initialization": |FunctionComposition expr: procedure to run after creating an instance|,
        "consolidation": |FunctionComposition expr: procedure to run after cloning an instance|
      }
    }
  }
}
```

Listing 2: The "template" definition of *DomainConcept* **data** in the Concept Hierarchy.

4.1 Definition of *DomainConcept* Properties

A concept's **properties** are the structural core of its description: each property records something that can be known about an instance of the concept.

Definition 4.1 (Property)**.** A *property* of a *DomainConcept* C is a named, typed element of the description of instances of C. It is *intrinsic* if it describes an instance of C in isolation, and *relational* if its *ValueDomain* relates that instance to one or more other *DomainConcept* instances.

The type of a property — the domain of values it may take — is always a ground *ValueDomain* type application (definition 5.3); this is the sense in which the Concept Hierarchy's data types are literally domains of values, and the reason the language names them *ValueDomains*. A relational property is expressed by giving it an `Instance`-typed *ValueDomain*, whose multiplicity and necessity are themselves carried by the *ValueDomain*: a bare `T` for a mandatory single value, `Optional<T>` for an optional one, and `Set<T>`, `List<T>`, or `Map<K, V>` for collections. These *ValueDomains* are not restricted to relational properties; they can also be used to type intrinsic properties. The type discipline is the subject of section 5; here we take them as given and describe how a property is *defined*.

Beyond its type, every property value in the Concept Hierarchy carries an intrinsic *confidence*: a degree in $[0, 1]$ recording how certain the system is that the value is correct, with 0 denoting complete ignorance and 1 certainty. Confidence is part of the data model itself rather than an auxiliary attribute added alongside the domain schema, so that a Concept Hierarchy never records a value as simply *true*, only as held with some degree of confidence. How a confidence value is fixed at measurement, and how the confidences of several values combine when one is computed from the others, are properties of a Concept Hierarchy's execution and are outside the scope of this document (subsection 1.2). Its decay over time is fixed statically, by the single property-definition keyword **confidenceHalfDecayTime** of subsubsection 4.1.4.

4.1.1 Defining A Property

A property is defined by an entry in the **properties** `JSON object`, mapping the property's name — a lowercase-initial `JSON string` — to its definition. That name is not merely local to the concept that defines it: it is the Concept Hierarchy's global handle on the property, and is constrained accordingly.

Definition 4.2 (Name uniqueness)**.** The names of *DomainConcept* **properties** and **functions** inhabit a single namespace spanning the entire Concept Hierarchy. No two of them — two properties, two functions, or a property and a function, whether defined on the same *DomainConcept* or on unrelated ones — may share a name.

Definition 4.2 guarantees that a property name determines its definition, and hence its **valueDomain**, with no reference to the concept on which it was defined. A name occurring on its own is therefore always resolvable to an expected type — whether as a key of the *properties* JSON object when a *DomainConcept* is instantiated, or as a link in an instance-property chain — with no enclosing type context needed to disambiguate it (subsection 8.2). The definition of properties takes one of two forms. The *shorthand* form is simply the JSON string naming the property's *ValueDomain*, with every other aspect of the property left at its default. The *explicit* form is a JSON object whose keys refine the property along the dimensions summarized below and laid out in Listing 3.

```
{
  |...|,
  "DomainConceptExampleWithProperties": {
    "directParents": ["Concept"],
    "data": {
      "properties": {
        "exampleProp1": "String",
        "explicitDefinitionOfExampleProp1": {
          "valueDomain": "String"
        },
        "explicitExampleProp2": {
          "description": |string: explanation of what the property describes|,
          "valueDomain": |string: ground ValueDomain (see Definitions 5.2 and 5.3)|,
          "static": |boolean: whether the same value is shared with all instances or not|,
          "constraint": |Boolean-resulting Function composition, subtype of property's ValueDomain,
    or a Variation: the constraint on the values of the property|,
          "hooks": {
            |string: ground Function (see Definitions 5.2 and 5.3)|: {
              |string: argument name of ground Function|: {
                 |"pre" or "post"|: |FunctionComposition expr: forward consistency propagation|,
                 |...|
              },
              |...|
            },
            |...|
          },
          "computations": [|FunctionComposition expr: backward consistency computation|, |...|],
          "confidenceHalfDecayTime": |Duration expr: time in which the confidence in the property
    value is halved|,
          "default": |prop. ValueDomain expr: sets unspecified property value during instantiation|
        },
        "_specializations": |discussed in subsection 4.2|
      },
      |...|
    }
  }
}
```

Listing 3: The "template" definition of *DomainConcept* **properties data** in the Concept Hierarchy.

The keys of an explicit property definition fall into four groups, according to what they govern: the property's *type and admissible values*, its *consistency* with related properties, its *confidence*, and two housekeeping concerns.

4.1.2 Type And Admissible Values

The **valueDomain** fixes the property's base type, as above. Within that type, **constraint** carves out a subset of admissible values, and **default** governs how an unspecified value is completed.

Definition 4.3 (Constraint)**.** A property's **constraint** restricts its value to a subset of its *ValueDomain* T, and is given in one of two forms. The *transparent* form is a FunctionCompositionRes<Boolean> expression evaluating to a Boolean, whose computation the Concept Hierarchy can inspect. The *opaque* form is an instantiation of Variation<T>, a *ValueDomain* denoting an implicit subset of T's values, whose membership computation the

Concept Hierarchy cannot see. A `JSON string` value of *Type* `TypeValue`, for example `"s:SubT"`, naming a subtype (definition 5.4) of `T` is a shorthand for the opaque form: it is converted to a `SubTypeVariation<T, SubT>` constraint[3].

A `constraint` may depend on the values of other properties of the same instance: a `Container`'s `contentVolume`, for example, may be constrained to lie between zero and that instance's `contentVolumeCapacity`. A `constraint` therefore refines, and is defined separately from, the property's `valueDomain`.

The `default` keyword decides, per property, whether an unspecified value is read under an open- or a closed-world assumption.

Definition 4.4 (Open-/closed-world property)**.** A property with no `default` is *open-world*: if an instance leaves it unspecified, the value is *unknown* (confidence 0). A property that specifies a `default` is *closed-world*: an unspecified value is completed with the default, at full confidence.

Under the open-world reading, an unspecified value means only that the value is *unknown*, not that it is absent; genuine optionality — a value that may legitimately not exist — is instead expressed in the type, through `Optional<T>`. A `default` is, in this sense, defeasible: it supplies a value only in the absence of more specific information, a point that becomes significant once defaults are specialized down the hierarchy (subsection 4.2). If a `default` value is used to fill a property's value, it will be checked against that property's enforced `constraint`.

4.1.3 Consistency

A property is rarely independent of the others. An `Object`'s `ownedBy` must name the `Person` whose `owns` lists it; a `Container`'s `contentVolume` must agree with the sum of volumes of what it contains. Two property-definition keywords maintain such relations, and they differ in direction. `hooks` propagate a change *forward*: when this property's value changes, the related properties are brought into agreement with the new value. `computations` work *backward*: they re-derive this property's value from the related properties, on demand rather than at the moment those properties change. The two are complementary rather than alternative, and a property may declare both. Between them, they let a Concept Hierarchy maintain the kinds of cross-property invariant that description logics express with a fixed vocabulary of constraint types — inverse, transitive, symmetric, and so on — but through arbitrary, user-defined *Function* procedures rather than a closed, built-in set.

Hooks. A property hook is attached to *positions* at which that property may be changed: the pair of a *Function* and one of that *Function*'s arguments. This is what the three levels of nesting under `hooks` record (Listing 3). The first key is a ground type application (definition 5.3) of a *Function* F. The second is the name of an argument a of F's `"interface"` (definition 7.1), and should be an argument declared `Modify` or `GetModify`, since an argument a *Function* only reads is not a position at which a change occurs[4]. The third is `"pre"` or `"post"`, each optional and each mapping to a `FunctionComposition` expression. A hook so written applies to an evaluation of F in which this property is the value supplied for a: the `"pre"` body runs before F's own procedure and the `"post"` body after it, so the former sees the property's value as it was and the latter as F modified it.

Attaching hooks per (*Function*, argument) pair, rather than per property, is what lets one property propagate differently according to how it was changed. An assignment to `owns`, replacing the whole list, and an append to `owns`, adding one element, are evaluations of different *Functions*, and the `ownedBy` of the `Objects` that must be corrected differs accordingly. A property changed by a *Function* for which it declares no hook propagates nothing.

Within a hook body, the implicit variables are the properties and functions of the concept, the instance on whose property the hook was triggered, and the remaining arguments of F (Table 12). The latter are what make the change itself visible: the value being appended to `owns` is an argument of the appending *Function*, and a hook maintaining `ownedBy` needs it.

A hook body carries one obligation that the Concept Hierarchy does not verify. It must *assert* the target state of the properties it maintains rather than apply an increment to their current state. The reason is that the number of times a hook body runs for one logical change is not fixed by the text: a maintained property may itself carry hooks, so one change may propagate along a chain and revisit a property more than once. A body that assigns

[3] `SubTypeVariation<T, SubT>` is a variation (subsection 5.6) accepting only values of `SubT`, which must be a subtype of `T`.

[4] While not enforced by a Concept Hierarchy, that hooks are not attached on `Get` arguments, we do not yet see a pattern in which a modeler would need attaching hooks on such *Function* arguments.

a computed target value is unaffected by this; a body that adds to, removes from, or otherwise adjusts a current value is not. Whether a given body satisfies the obligation is a modeler's responsibility.

Computations. A property's `computations` value is a list of `FunctionComposition` expressions. Each element is one *derivation* of the property: an independent way of arriving at its value from other properties. A property with several derivations is one that may be known in several ways — a mass from a density and a volume, or by summing up the mass of its contents — and the list records them all rather than requiring the modeler to fold them into one body.

Which property a derivation computes is fixed by where it is written, and not restated in the body; what a derivation may read is fixed by the implicit variables in scope, which are the instance, and its properties and functions. A derivation is not obliged to produce a value. A derivation whose inputs are unknown, or whose applicability condition fails, may *abstain*, and abstention is not an error, but an ordinary outcome recognized by no `Return<T>` *Function* being evaluated. This is why the elements of the list are typed `FunctionComposition` and not by a *ValueDomain* committing them to a result of the property's own `valueDomain`: a body that must yield a value of that *Type* has no way to decline. Neither does a derivation report its result through a designated output variable. The value a derivation produces, and the confidence that value carries, are determined by the composition's own evaluation and by the confidences of the properties it read; nothing further need be, and nothing further may be, written in the body to record them.

A read of a property value is the only thing that can cause the evaluation of a derivation. Whether a derivation is re-evaluated depends on whether the values that are read (accessed) in the derivation have changed since the derivation last ran. The result of a property read is the value of the derivation with the highest confidence and, in case of a tie, the derivation with the lowest index. The measured/assigned value is the derivation with index 0, and the index i of the other derivations is the index $i-1$ in the list of `computations` of the property.

Both `hooks` and `computations` keywords are specializable, and both are implementation-facing rather than contract-bearing, so a subconcept may replace or cancel either freely (definition 4.7). The Concept Hierarchy composes no inherited body with its replacement, so a subconcept replacing a parent's hook or derivation takes on whatever that body maintained (subsubsection 4.2.7).

4.1.4 Confidence

A property may set `confidenceHalfDecayTime`, a `Duration` value giving the time over which confidence in the property's value decays to half of its value at the moment of last measurement; when left unspecified, the property's confidence does not decay. Writing Δ_p for the declared duration, c_p^{ref} for the confidence recorded at the last measurement of a property p, and t_p^{ref} for the time of that measurement, the confidence of p at a time $t \geq t_p^{\text{ref}}$ is

$$c_p(t) \;=\; c_p^{\text{ref}} \cdot 2^{-\frac{t - t_p^{\text{ref}}}{\Delta_p}},$$

and an unspecified `confidenceHalfDecayTime` is read as $\Delta_p = \infty$, for which the expression is constant. Because `confidenceHalfDecayTime` is specializable (subsection 4.2), one property may decay at different rates over its lifetime depending on the instance's concepts; the expression above governs one segment, at the rate then in force.

4.1.5 Housekeeping

Finally, `static` marks a property whose value is shared across all instances of the concept — so that a change to it is visible from every instance — in contrast to the default, per-instance reading; and `description` carries free text documenting what the property represents in the domain, with no effect on meaning.

4.2 Specializing *DomainConcept* Property Definition Data

A property is *defined* exactly once, at the concept where its data first appear. As the hierarchy is refined into more specific concepts, however, the *behavior* of that property — its admissible values, its default, its consistency mechanisms — must frequently be refined as well; and the value used for a concept's own direct instances may need to differ from the value its subconcepts inherit. The Concept Hierarchy expresses this refinement through a *property specialization* mechanism that acts, at every concept, over two independent scopes.

Definition 4.5 (Specialization scope)**.** At every concept, each specializable keyword of a property is specialized independently across two scopes. The ***forThis*** scope supplies the value used for direct instances of *this* concept; the ***forSubconcepts*** scope supplies the value inherited by, and further specializable in, this concept's subconcepts.

4.2.1 Specializable And Fixed Keywords

Five of a property's definition keywords may be specialized, independently across the two scopes: `constraint`, `hooks`, `computations`, `confidenceHalfDecayTime`, and `default`.[5] The remaining three keywords are *fixed* at the property's definition site for the lifetime of the hierarchy and admit no specialization: `valueDomain` (the property's base type), `static` (whether the value is shared across all instances or held per instance), and `description` (the textual documentation of the property).

4.2.2 Specialization Values

At either scope, a specializable keyword does not merely hold a value or not; it occupies one of four *specialization values*, which distinguish *how* a value is obtained — written directly, canceled, or inherited, and if inherited, from where.

Definition 4.6 (Specialization value)**.** At a given scope, a specializable keyword takes one of four specialization values:

- Set Value — an explicit value for the keyword is written;
- No Value — the keyword is explicitly set to undefined, canceling (declining) any inherited value;
- Implicit Get — the keyword is omitted, and its value is resolved from the concept's parents;
- From Parent — the keyword's value is taken from one explicitly named parent.

Two of these presuppose a parent to inherit from, and so are available only on *inherited* properties: Implicit Get and From Parent. On a property at the concept that *defines* it, only Set Value and No Value are meaningful. The states differ further in their applicability to the monotonic keyword `constraint`, summarized in Table 3: on an inherited property, only Set Value and Implicit Get are valid for `constraint`, for reasons developed in subsubsection 4.2.4.

State	Meaning	Valid for `constraint`?
Set Value	an explicit value for the definition keyword is written	yes
No Value	the definition keyword's value is set to not-defined	only in `_forThis`, at the definition concept
Implicit Get	the definition keyword is unspecified; its value is resolved from parent(s)	yes — resolves to a *conjunction*, not a selection
From Parent	explicit reference to a named parent from which to get the definition keyword's value	no

Table 3: Specialization value states for property keyword specialization, and their validity for the monotonic keyword `constraint`. All four states are unrestricted for the four non-monotonic keywords (`default`, `confidenceHalfDecayTime`, `hooks`, `computations`); see subsubsection 4.2.4.

4.2.3 Syntax

Specializations are written under the `_specializations` keyword within a *DomainConcept*'s `properties` `JSON object`. Its leading `_` prevents any collision with property names, which always begin with a lowercase letter. Its value is a nested `JSON object` addressed along two paths, one per scope:

[5] Unlike the other four, `constraint` is subject to a monotonicity restriction: it may only be narrowed, never canceled, in subconcepts, with a single exception restricted to the property's definition site (subsubsection 4.2.4.)

```
_specializations -> propName -> propKeyword -> |specializationValue|
_specializations -> _forThis -> propName -> propKeyword -> |specializationValue|
```

The top-level path carries ***forSubconcepts*** values, and its `propName` keys are restricted to properties *inherited* from a parent. The nested **`_forThis`** path — likewise `_`-prefixed to keep clear of property names — carries ***forThis*** values, and its `propName` keys may name both inherited properties and properties defined locally at this concept. For a property *defined* at this concept, the value written at its definition site already *is* its ***forSubconcepts*** specialization; consequently, a defined property's keywords may appear only along the **`_forThis`** path, never in the top-level **`_specializations`** path.

Each `|specializationValue|` is either an explicit value (a Set Value) or one of two reserved `inheritFrom:` `JSON string` forms, which encode the two inheritance states:

```
"definitionKeyword": "inheritFrom:"        // No Value (cancels inheritance)
"definitionKeyword": "inheritFrom:<Name>"  // From Parent <Name>
```

The two forms differ only in the presence of a parent name yet denote opposite operations: the bare form makes a value *undefined*, while the named form *selects* a value.

The fourth state, Implicit Get, has no explicit syntax: it is what a keyword's *absence* denotes — but the two paths interpret absence differently. A keyword absent from the top-level path is an Implicit Get: its ***forSubconcepts*** value is resolved against the parents, as described below. A keyword absent from **`_forThis`**, by contrast, does *not* consult any parent; the ***forThis*** scope simply *mirrors* whatever value was established for ***forSubconcepts*** at this same concept, whether that value came from a top-level specialization or from the property's definition site. This mirroring is not itself an Implicit Get, since it resolves against no parent and reuses a value already fixed locally.

When a concept specializes a keyword for ***forSubconcepts*** only, but wants ***forThis*** resolved *independently* against the parents rather than mirroring that local value, it must say so explicitly, with a third reserved form, `"inheritFrom:parents"`, valid only inside **`_forThis`**:

```
"_specializations":{"_forThis": {"propertyName": {"definitionKeyword": "inheritFrom:parents"}}}
// the forThis scope value resolves via Implicit Get, ignoring the local forSubconcepts value
```

Since absence at the top level already denotes Implicit Get unambiguously, `"inheritFrom:parents"` is not permitted there and is meaningful only inside **`_forThis`**. The explicit `inheritFrom:<Name>` form is also available within **`_forThis`**, letting ***forThis*** select a named parent's value directly, rather than mirroring the local ***forSubconcepts*** value or deferring to ordinary Implicit Get resolution via `inheritFrom:parents`.

4.2.4 Resolution and specialization regimes.

The four inheritance-bearing situations — Implicit Get in either scope, and `inheritFrom:parents` in ***forThis*** — must ultimately be *resolved* against the concept's parents. How they resolve depends on the keyword, and the five specializable keywords are split into two regimes governed by genuinely different rules, not merely different permissions.

Definition 4.7 (Specialization regime)**.** **`constraint`** is *contract-bearing*: because a subconcept's direct instances remain, transitively, instances of every parent concept, an instance must satisfy *every* parent's constraint at once. **`constraint`** therefore resolves *monotonically*, as the *conjunction* of all contributing parents' constraints, never a selection among them. The remaining four keywords — **`default`**, **`confidenceHalfDecayTime`**, **`hooks`**, and **`computations`** — are *implementation-facing*: nothing in the Concept Hierarchy reasons about an instance's admissible values through a statically held parent reference for these. Therefore, each resolves *non-monotonically*, to a single value freely overwritable or cancellable in any subconcept.

Concretely, for the four non-monotonic keywords, an Implicit Get (or an `inheritFrom:parents` in ***forThis***) resolves against the direct parents' ***forSubconcepts*** values by *selection*: if exactly one parent supplies a value other than No Value, that value is used; if none does, the result is No Value; if more than one does, resolution is ambiguous and reported as an error. The `inheritFrom:<Name>` form exists precisely to resolve that ambiguity under multiple inheritance, and is available, in either scope, for the non-monotonic keywords only.

For `constraint`, the same inheritance situations resolve instead by *conjunction*: every contributing parent's resolved ***forSubconcepts*** constraint is combined with a logical AND. Because a conjunction is commutative and defined for any number of operands, this resolution is never ambiguous, even though many parents contribute under multiple inheritance. Consequently, `inheritFrom:<Name>` is not a meaningful disambiguator for `constraint` and is not a valid specialization value for it (Table 4); `"inheritFrom:parents"` remains available in `_forThis` for `constraint`, where it triggers this same conjunction. If no parent contributes, the conjunction is vacuous, and the specialized value is undefined, exactly as in the non-monotonic case.

	`constraint` (monotonic)	other four keywords (non-monotonic)
Explicit SET VALUE at a subconcept	narrows: conjoined with, never replacing the constraint inherited from every parent	replaces the value inherited from every parent
IMPLICIT GET: one contributing parent	that parent's resolved value	that parent's resolved value
IMPLICIT GET: multiple contributing parents	conjunction of all — always well-defined, never ambiguous	ambiguous ⇒ error; to be disambiguated via `inheritFrom:<Name>`
IMPLICIT GET: no contributing parent	no specialized value	no defined value from any parent
FROM PARENT (`inheritFrom:<Name>`)	not permitted: naming one parent would silently drop every other parent's constraint	permitted; selects exactly the named parent's value
NO VALUE (`inheritFrom:`)	not permitted, in either scope, at any subconcept; except in ***forThis*** scope of the property's defining concept	permitted, in either scope, for both defined and inherited properties/functions

Table 4: Resolution semantics contrasted between the monotonic keyword `constraint` and the four non-monotonic keywords.

The single exception for `constraint` — NO VALUE permitted in `_forThis` at the property's *defining* concept — has a simple justification. At the concept where a property is defined, no parent constraint yet exists, so there is nothing to narrow; and that concept's own direct instances may legitimately be exempted from the constraint it imposes on its subconcepts, by specifying `"inheritFrom:"`, without violating subsumption.[6]

4.2.5 Specialization Types

The presence or absence of a keyword along the top-level path, combined with its presence, absence, or use of `"inheritFrom:parents"` within `_forThis`, yields one of five *specialization types*, as per Table 5.

Specialization Type	Top-level Data	`_forThis` Data	***forSubconcepts*** Value	***forThis*** Value
forBothThisAndSubconcepts	explicit V	absent	V	V (mirrored)
forThisAndForSubconcepts	explicit V	explicit W	V	W
onlyForSubconcepts	explicit V	`inheritFrom:parents`	V	IMPLICIT GET
onlyForThis	absent	explicit W	IMPLICIT GET	W
noSpecialization	absent	absent	IMPLICIT GET	IMPLICIT GET (mirrored)

Table 5: Mapping from `_specializations` syntax to the five specialization types and their resolved scope values. V and W denote any explicit specialization value, i.e. SET VALUE, NO VALUE (via `inheritFrom:`), or FROM PARENT (via `inheritFrom:<Name>`).

[6] This is the only case in which `_forThis` for `constraint` may cancel rather than narrow. Every other specialization of `constraint` yields a constraint as narrow as, or narrower than, that of the parent concept(s).

The five types are best read as three pairs of behaviors plus the trivial case. In **forBothThisAndSubconcepts** and **noSpecialization**, ***forThis*** is not chosen independently but simply mirrors whatever ***forSubconcepts*** resolves to, explicit or implicit. **onlyForSubconcepts** deliberately decouples the two scopes despite ***forSubconcepts*** having an explicit value, because `inheritFrom:parents` forces ***forThis*** to resolve independently against the parents. **onlyForThis** is the converse: ***forSubconcepts*** is IMPLICIT GET while ***forThis*** is pinned to an explicit value. **forThisAndForSubconcepts** is the fully decoupled case, both scopes explicit and typically distinct. Within any row, V and W may independently be any of the three explicit states; in particular, a NO VALUE for V in **onlyForSubconcepts** does not pass the inherited specialization on to subconcepts,[7] while **forThisAndForSubconcepts** lets V and W combine states freely — for example a SET VALUE for subconcepts paired with a NO VALUE for this concept's own instances.

4.2.6 Example

Listing 4 exercises all five types on a single property `p`, defined on `X` and specialized down through `Y` (a direct subconcept of `X`) and `Z` (a subconcept of `Y`). `X` sets a **`constraint`** and **`default`** shared by both scopes, then overrides **`default`** for its own direct instances via **`_forThis`**. `Y` narrows **`constraint`** for its subconcepts while requesting, via `inheritFrom:parents`, that its own direct instances resolve **`constraint`** independently against `X` rather than mirror the narrowed subconcept value; it also inherits **`default`** from `X` explicitly, for instances of `Y`, rather than mirroring the IMPLICITGET value from the ***forSubconcepts*** scope. `Z` does not specialize the **`constraint`** of `p` at all. Table 6 makes the trace of keyword specializations explicit.

```
{
  "X": { "data": { "properties": {
    "p": {
      "valueDomain": "Integer",
      "constraint": { "Interval<Integer>":  [0, 5] },
      "default": 1
    },
    "_specializations": {
      "_forThis": { "p": { "default": 4 } }
    }
  }}},
  "Y": { "directParents": ["X"], "data": { "properties": {
    "_specializations": {
      "p": { "constraint": { "Interval<Integer>":  [1, 5] } },
      "_forThis": { "p": { "constraint": "inheritFrom:parents", "default": "inheritFrom:X" } }
    }
  }}},
  "Z": { "directParents": ["Y"], "data": { "properties": {
    "_specializations": {
      "p": { "default": 5 }
    }
}}}}
```

Listing 4: Exemplary property specialization syntax showcasing the five specialization types.

A direct instance of `Y` admits the value 0, while an instance of `Z` does not, because `Y`'s `inheritFrom:parents` sends its own instances back to `X`'s $[0, 5]$ while its subconcepts inherit the narrowed $[1, 5]$. Subsumption is nonetheless preserved: `Z`'s instances satisfy $[1, 5] \subseteq [0, 5]$, and every concept's resolved ***forThis*** **`default`** (4, 1, and 5 respectively) lies within its resolved ***forThis*** **`constraint`**.

4.2.7 Subsumption And Monotonicity

subsubsection 4.2.4 and Table 4 give the resolution mechanics that separate **`constraint`** from the four non-monotonic keywords; this paragraph gives the rationale behind the split. It mirrors the distinction between a

[7] Not applicable to **`constraint`**, for which NO VALUE on an inherited property is invalid under every specialization type; see Table 4.

		Written in			Resolved value	
	Keyword	`_specializations`	`_forThis`	Specialization type	***forSubconcepts***	***forThis***
X	`constraint`	$[0, 5]$ (def. site)	—	**forBothThis-AndSubconcepts**	$[0, 5]$	$[0, 5]$ (MIR.)
	`default`	1 (def. site)	4	**forThisAnd-ForSubconcepts**	1	4
Y	`constraint`	$[1, 5]$	`inheritFrom :parents`	**onlyForSubconcepts**	$[0, 5] \wedge [1, 5]$ $\equiv [1, 5]$ (CONJ.)	$[0, 5]$ (CONJ.)
	`default`	—	`inheritFrom:X`	**onlyForThis**	1 (SEL.)	1
Z	`constraint`	—	—	**noSpecialization**	$[1, 5]$ (CONJ.)	$[1, 5]$ (MIR.)
	`default`	5	—	**forBothThis-AndSubconcepts**	5	5 (MIR.)

Table 6: Trace of Listing 4: how the specialization data that is written at each concept resolves, per scope, for the two specialized keywords of property `p`. The *written* columns reproduce the listing, with — marking an absent keyword and *def. site* a value written at `p`'s definition site, which *is* its ***forSubconcepts*** specialization. The *resolved* columns are derived by subsubsection 4.2.4: MIR. marks a ***forThis*** value mirrored from the local ***forSubconcepts*** value, CONJ. a conjunction over the contributing parents (`constraint`, monotonic), and SEL. a selection of a single parent's value (`default`, non-monotonic). $[a, b]$ abbreviates `{Interval<Integer>: [a, b]}`.

routine's *contract* and its *implementation* in Design-by-Contract systems [Meyer(1992)]. `constraint` is contract-bearing, so an instance must satisfy the conjunction of constraints inherited from *all* parents, which is precisely what preserves the subsumption relation between a concept and its subconcepts. `default`, `confidenceHalfDecay-Time`, `hooks`, and `computations` are implementation-facing, and may therefore be overwritten or canceled freely in any subconcept:

- `default` is, by design, a defeasible value in the sense of classical non-monotonic reasoning [Reiter(1980), McCarthy(1980)]: it supplies a value only in the absence of more specific information and is expected to be superseded by it. A subconcept withdrawing or replacing an inherited `default` is the mechanism working as intended, since no code holding a reference to a parent concept is entitled to assume that an unspecified property resolves to that parent's default.
- `confidenceHalfDecayTime` may differ freely per concept, reflecting that decay dynamics are an empirical property of the concept being modeled rather than a subsumption-relevant guarantee. This freedom presupposes that `confidenceHalfDecayTime` is always resolved dynamically, against an instance's actual, most-derived concept, at the point confidence is decayed; a code path that instead resolved or cached this value statically off a supertype reference would silently ignore a subconcept's override wherever only the supertype is statically known.
- `hooks` and `computations` are implementation and may be freely reimplemented in a subconcept, analogously to overriding a virtual method. The Concept Hierarchy performs no automatic composition of an inherited hook or computation body with its replacement, so any cross-property or side-effecting behavior enforced by a parent's hook is silently lost in a subconcept that replaces it without independently re-implementing that behavior.[8]

Property specialization in the Concept Hierarchy is therefore monotonic exactly where subsumption requires it (`constraint`) and freely non-monotonic elsewhere, by design.

[8] The same pitfall as omitting a call to `super()` in conventional object-oriented inheritance.

4.2.8 Interaction With Open- And Closed-World Reading

The open-/closed-world status of a property (definition 4.4) is fixed not by the definition-site **default** but by its *resolved* value at the concept in question: a resolved **default** in the SET VALUE state completes an unspecified value (closed-world), whereas a resolved NO VALUE — whether never set, or canceled via `inheritFrom:` — leaves it genuinely unknown (open-world). Whether a value must be present at all is a separate, type-level matter, expressed through `Optional<T>` rather than through the specialization mechanism described here.

4.3 Definition of *DomainConcept* Functions

Where a concept's **properties** give its structural description, its **functions** attach behavior to its instances — the object-oriented half of a *DomainConcept*. A function is defined by an entry in the **functions** `JSON object`, mapping the function's name — a lowercase-initial `JSON string` — to its definition. Function names share the single namespace of definition 4.2, and so must differ from every property and (other) function name of the Concept Hierarchy. A function's behavior is a procedure, supplied as its **default** value; that value is a `CustomFunction` expression, the general mechanism by which a procedure is written as data, defined in section 7.

A function definition admits far fewer keys than a property definition: only **description**, **static**, and **default** (Listing 5). The property-specific machinery — **valueDomain**, **constraint**, **hooks**, **computations**, and **confidenceHalfDecayTime** — has no counterpart for functions, which is precisely why the Concept Hierarchy declares **functions** separately from **properties** rather than folding them together.

```
{
  |...|,
  "DomainConceptExampleWithFunctions": {
    "directParents": ["Concept"],
    "data": {
      "functions": {
        "exampleFunc1": {
          "description": |string: explanation of what the function performs|,
          "static": |boolean: whether the same value is shared with all instances or not|,
          "default": |CustomFunction expr: sets unspecified function value during instantiation|
        },
        "shorthandFunc": |CustomFunction expr: interpreted as a static function with this default|,
        "_specializations": |discussed in subsection 4.4|
      },
      |...|
    }
  }
}
```

Listing 5: The "template" definition of *DomainConcept* **functions data** in the Concept Hierarchy.

As syntactic sugar, the definition of a function can directly be a `CustomFunction` instantiation rather than an object of the keys above like for `exampleFunc1`. It is then interpreted as a non-**static** function whose **default** is that `CustomFunction` value, as for `shorthandFunc` above.

4.4 Specializing *DomainConcept* Function Definition Data

Because **default** is the only specializable keyword of a function, function specialization is a strict simplification of the property case of subsection 4.2: the same two scopes, the same four specialization values, and the same non-monotonic resolution apply, but to a single keyword. The only addition is a shorthand: a bare `CustomFunction` expression written under a function's name — along either path — is read as a specialization of that function's **default**, sparing the explicit `"default"` key (Listing 6).

```
{
  "X": { "data": { "functions": {
    "f": { "static": true },
    "_specializations": {
      "_forThis": {
```

```
        "f": |CustomFunction expr: specialized default value for direct instances|
      }
    }
  }}},
  "Y": { "directParents": ["X"], "data": { "functions": {
    "_specializations": {
      "f": |CustomFunction expr: specialized default value for subconcept instances|,
      "_forThis": { "f": "inheritFrom:parents" }  // no value to inherit from X => default is unset
    }
}}}}
```

Listing 6: The shorthand function specialization syntax.

4.5 Definition of *DomainConcept* Management

Some operations on an instance leave it in a state that is momentarily inconsistent with the rest of the database — not through any error, but because the operation touches one instance at a time while the invariants it must respect span several. A concept's `management` functions are the language's *hook* for restoring consistency once such an operation has completed.

Definition 4.8 (Management functions)**.** A *DomainConcept*'s `management` defines two *Function* procedures, each a `FunctionComposition` expression (section 7): `initialization`, run after an instance of the concept is *created*, and `consolidation`, run after an instance is *cloned*. The former restores consistency of a newly created, possibly underspecified instance; the latter restores consistency of an instance's underlying implementation data after a clone.

```
{
  |...|,
  "DomainConceptExampleWithManagement": {
    "data": {
      "management": {
        "initialization": |FunctionComposition expr: after creating an instance of this concept|,
        "consolidation": |FunctionComposition expr: after cloning an instance of this concept|
      },
      |...|
    }
  }
}
```

Listing 7: The "template" definition of *DomainConcept* `management` `data`.

4.5.1 Initialization

`initialization` completes a newly created instance whose specification, being local, may leave related invariants unmet. Listing 8 illustrates the canonical case, an inverse relation: a `Person` `owns` `Objects`, and an `Object` may in turn be `ownedBy` a `Person` (the `Optional<T>` *ValueDomain* marks `ownedBy` as a possible, not mandatory, value). The instance `JamesHetfield` sets only the forward direction of this relation.

```
{
  "concepts": {
    "Person": { "data": {
      "properties": { "name": "String", "owns": "List<Instance<Object>>",
                      "favoriteObject": "Optional<Reference<Instance<Object>>>" },
      "management": {
        "initialization": |FunctionComposition expr: initializes Object.ownedBy for owned Objects|,
        "consolidation": |FunctionComposition expr: corrects internal data of Reference clone|
      }
    }},
```

```
    "Object": { "data": { "properties": { "ownedBy": "Optional<Instance<Person>>" } } }
  },
  "instances": {
    "JamesHetfield": {
      "InstanceBase":  {
        "instanceName":  "s:JamesHetfield",
        "concepts":  ["s:Person"],
        "properties":  {
          "name": "s:James Hetfield", "owns": ["CustomFlyingV"], "favoriteObject": "CustomFlyingV"
        }
      }
    },
    "CustomFlyingV": { "InstanceBase":  {
      "instanceName": "s:CustomFlyingV", "concepts": ["s:Object"], "properties": {}
    }}
  }
}
```

Listing 8: "Ownership" Concept Hierarchy demonstrating the need for **management** *Functions*.

The instance data records only that `JamesHetfield.owns` contains `CustomFlyingV`; nothing sets the inverse, `CustomFlyingV.ownedBy`. `Person`'s **initialization** *Function* closes this gap by iterating over the `owns` collection and setting `ownedBy` on each `Object` it names — the automatic maintenance of an inverse relation upon instance creation familiar from frame-based ontology design [Noy and McGuinness(2001)]. Without it, the two instances would remain mutually inconsistent with respect to the bidirectional relation. During a Concept Hierarchy's execution, the **hooks** are responsible for maintaining the consistency of the relation.

4.5.2 Consolidation

Cloning raises a structurally similar problem, but triggered by duplication rather than creation. Cloning an instance at a chosen moment is useful for *what-if* reasoning — planning, look-ahead, or simulating alternative outcomes — and demands that the clone be internally consistent and fully detached, so that no edit to the clone is visible in the original. When a clone is made, every *ValueDomain* duplicates its own implementation data; the `favoriteObject` property of Listing 8, of type `Reference<T>`, is duplicated along with the rest. Duplication alone does not suffice for a reference: in a `C++` implementation backing `Reference<T>` with a `pointer`, a naively copied pointer still addresses the *original* instance rather than its clone. Nor can a more careful clone fix this locally, because at the instant `Reference<T>` is cloned, the instance it should point to may not yet exist in cloned form — that instance is cloned independently, as part of the same overall operation. **consolidation** resolves this by re-pointing such references to the cloned state once the entire clone operation has finished.

5 Definition of *ValueDomains*

ValueDomains are the data types of the Concept Hierarchy: they constrain the values a *DomainConcept* **property** may take, and each is an *implicit* definition of a set of values. The definition is implicit because the Concept Hierarchy has no access to a *ValueDomain*'s implementation; the set becomes explicit only to the application designer, i.e. the programmer realizing the *ValueDomain* in the underlying language, `C++` for example. A *ValueDomain* definition in the Concept Hierarchy is therefore an interface to a set of values.

Many *ValueDomains* describe not a single type but a *family* of related types — a `Container` parameterized by the type of object it holds, say. To describe such families, a *ValueDomain* may declare one or more *template parameters*. Its **data** definition has three parts, shown in Listing 9: a **templateContext** declaring the *ValueDomain*'s own template parameters and the arguments it substitutes into its parents' parameters, a **defaultSerialization** associating it with a `JSON` primitive type, and an **instantiation** defining which `JSON` values are validated as its instances.

```
{
  |...|,
  "ValueDomainExample": {  "data": {
```

```
    "templateContext": {
      "order": |array of strings: template parameter name list (suffix '...' marks as variadic)|,
      |string: template parameter name|: |string: template parameter constraint formula|,
      |...|,
      "variadicGroupIdentifiers": {
        |string: variadic template parameter name (without '...')|: |string regex [!$]*: empty ok|,
        |...|
      },
      "substitution": {
        |string: <ParentConceptName>:<TemplateParameterNameOfParentConcept>|: |template argument:
    value to substitute for the parameter|,
        |...|
      }
    },
    "defaultSerialization": |null, "null", "boolean", "integer", "number", "string"|,
    "instantiation": |custom JSON schema defining the structure of the JSON value to be interpreted
    as this ValueDomain|
  }},
  "ValueDomainExampleWithShorthandtemplateContext": {  "data": {
    "templateContext": |array of strings: this is interpreted as the template parameter order|
  }},
  "ValueDomainExampleWithTemplateSpecificInstantiation": {  "data": {
    "instantiation": [
      [[|constraint formula of first template parameter|, |...|], |custom jsonschema|],
      |...|
    ]
  }}
}
```

Listing 9: The "template" definition of *ValueDomain* **data**.

Definition 5.1 (Parameterized *ValueDomain*)**.** A *ValueDomain* is *parameterized* if it declares template parameters in its **templateContext** definition.

Definition 5.2 (Type Application)**.** A *type application* is a *ValueDomain* or *DomainConcept* name, together with template arguments associated with every template parameter it declares, supplied using the type application brackets <> if it is a parameterized *ValueDomain*, or with no brackets otherwise. We refer to a type application as an *applied* or *type-applied* *ValueDomain* when its name resolves to a *ValueDomain*.

Definition 5.3 (Ground Type Application)**.** A type application is *ground* (or *fully applied*) if it contains no free template variables. Every *DomainConcept* and every non-parameterized *ValueDomain* is trivially a ground type application, having no template parameters to leave unresolved.

Definition 5.4 (Type)**.** A *Type* is a ground type application. The *Type* hierarchy is the parent relation over *Types*: *Types* are ordered by **directParents** and, for parameterized *ValueDomains*, by template substitution.

Definition 5.5 (Template Parameter)**.** A *template parameter*[9] is a formal placeholder declared by a *ValueDomain* definition, over which the *ValueDomain* is universally quantified, subject to a constraint formula restricting its admissible arguments. A template parameter is either:

- a *type parameter*, ranging over *Types* (definition 5.4), or
- a *literal parameter*, ranging over a literal domain (`integer`, `number`, `boolean`, or `string`).

Following `C++` terminology, we also call the latter a *non-type template parameter*.

[9] We refer to this parameterization mechanism as *templates*, following the terminology of C++, to emphasize that Concept Hierarchy instantiates parameterized *ValueDomains* via monomorphization: each distinct combination of type arguments compiles to its own ground *Type*, as in `C++`, rather than being erased to a shared runtime representation as in, e.g., Java generics.

Definition 5.6 (Template Argument)**.** A *template argument* is the *Type* or literal value supplied in place of a template parameter at the point where a parameterized *ValueDomain* is used: a *Type argument* for a type parameter, or a *literal argument* for a non-type template parameter.

A *ValueDomain*'s **`templateContext`** may declare template parameters, a substitution into a parameterized parent, or both. Declaring template parameters is only meaningful if the *ValueDomain* is itself parameterized; providing a substitution is meaningful whenever the *ValueDomain* has a parameterized parent, independent of whether the *ValueDomain* itself is parameterized.

5.1 Template Parameter Constraint Formulae

A template parameter's value is either a *Type* drawn from the Concept Hierarchy or a literal scalar of one of four primitive domains. The constraint language of this subsubsection allows the author of a *ValueDomain* to restrict the admissible values of each template parameter; Listing 10 presents its syntax, and the paragraphs below present its semantics and static well-formedness conditions.

```
// No variadic group constraint: all values of the variadic group must satisfy the same constraint!
// Do not differentiate syntactically the places where Unconstrained, i.e. '', is allowed; check semantically
// The canonical serialization form separates elements with ', '; the space after the comma is intentional
TemplateArgumentConstraintFormula
        ::= (ws '"' templateConstraintFormula '"' ws) | LiteralValue
templateConstraintFormula
        ::= templateConstraintOperator | templateConstraint
templateConstraintOperator
        ::= ((('And' | 'Or') '(' templateConstraintFormula (', ' templateConstraintFormula)* ')') | ('Not(' templateConstraintFormula ')'))
templateConstraint
        ::= literalTemplateConstraint | typeTemplateConstraint | ''
literalTemplateConstraint
        ::= ('Literal:' ('boolean' | 'integer' | 'number' | 'string')) | literalValue
typeTemplateConstraint
        ::= ((appliedTypeName '*'?) | ('^' appliedTypeName '*'?) | (appliedTypeName '.'))
appliedTypeName
        ::= upperCaseName ('<' (templateConstraintFormula (', ' templateConstraintFormula)*) '>')?
LiteralValue
        ::= ws (boolean | number | ('"\\"' character* '\\""')) ws
literalValue
        ::= boolean | number | '\\"' character* '\\"'
upperCaseName
        ::= [A-Z] ([A-Z] | [a-z] | digit | '_')*
```

Listing 10: The `EBNF` grammar for the definition of template parameter constraints. `ws` is a whitespace consuming rule. Uppercase-starting production rules handle `ws` tokens; lowercase-starting production rules are used inside `JSON string` values.

5.1.1 The *unconstrained* Formula

The empty production `templateConstraint ::=''` denotes the *unconstrained* formula, written as the absence of any token, and is semantically the universally true predicate: every *Type* and every literal value satisfies it. It is the sort-polymorphic top element $\top$ of the constraint algebra (section 5.1.4), and is used where a parameter is left unrestricted — for example, the second slot of the constraint formula `Pair<Number, >`, for a `Pair` *ValueDomain* with two template parameters. It may not be used at the point where a template parameter's own constraint is declared, but is available in every other constraint formula position, such as the template-parameter-dependent **`instantiation`** schemas of subsection 5.4.

5.1.2 Literal Constraints

A *domain constraint* `Literal:`s for $s \in \{$`boolean`, `integer`, `number`, `string`$\}$ restricts a template argument to the named scalar domain without pinning a particular value, admitting any boolean, any integer, any floating-point number, or any string respectively. A *value constraint* is a concrete literal — `true`, `false`, a decimal integer, a floating-point number, or a double-quoted string — and restricts the argument to exactly that one value; a value constraint for a value of literal domain s implies the domain constraint `Literal:`s, but not vice versa. At the constraint-definition point, a value constraint may be written either as a bare `JSON` literal (e.g. the `JSON number 3`) or as a quoted `string` formula containing it (e.g. `"3"`); the two are semantically equivalent, with no preference between them[10]. This shorthand applies only at the top level of a constraint definition: literal values inside a quoted formula, e.g. inside `Not(...)`, always use the `literalValue` production.

[10]The implementation normalizes a bare literal to its quoted-formula form before further processing.

5.1.3 Type Constraints: The Five Hierarchy Operators

The constraint for a type template parameter is specified with one of five *hierarchy operators*, summarized in Table 7. Each takes a named concept T as its reference point and defines a set of *Types* relative to the subsumption relation $\sqsubset$ of the hierarchy, where $\mathit{instantiable}(C)$ indicates that C is not declared abstract.

Syntax	Operator	Admitted set of *Types*
T	Descendants	$\{\, C \mid C \sqsubseteq T \,\}$
$T*$	NonAbstractDescendants	$\{\, C \mid C \sqsubseteq T,\ \mathit{instantiable}(C) \,\}$
$T.$	Self	$\{\, T \,\}$
$\hat{}T$	Ascendants	$\{\, C \mid T \sqsubset C \,\}$
$\hat{}T*$	NonAbstractAscendants	$\{\, C \mid T \sqsubset C,\ \mathit{instantiable}(C) \,\}$

Table 7: The five hierarchy operators and the sets of *Types* they admit. T denotes the reference concept; $\sqsubset$ is the subsumption relation of the hierarchy; $\mathit{instantiable}(C)$ holds when C is not declared abstract.

The operators vary along three axes. The bare form T and its starred variant traverse *downward* and include the anchor T itself ($\sqsubseteq$), whereas the caret forms traverse *upward* and exclude it ($\sqsubset$); the starred variants of either direction additionally drop the abstract, non-instantiable *Types*; and the dot form $T.$ is a *nominal* constraint admitting the named concept alone, used for example where a template argument is to be fixed exactly. The bare form applied to the hierarchy root, `"Concept"`, is the *universal type constraint*: it admits any *Type* whatever its position in the hierarchy, and is $\top$ read in the type sort.

5.1.4 Boolean Operators and Constraint Sorts

The operators `And`, `Or`, and `Not` compose constraints using classical propositional logic and are freely nestable, whether within one another or within the template-argument sub-constraints of subsubsection 5.1.5. `And(`F_1`, ..., `F_n`)` admits a value satisfying every sub-formula, `Or(`F_1`, ..., `F_n`)` one satisfying at least one, and `Not(`F`)` one satisfying none.

The operators are overloaded across all constraint sorts: `And(Animal, Not(Predator))` intersects two type-sort constraints to admit any *Type* descendant of `Animal` but not of `Predator`, while `And(Literal:integer, Not(3))` intersects two integer-sort constraints to admit any integer other than three. A static *sort discipline* governs this overloading: every formula has a *constraint sort* $\sigma \in \{\mathsf{type}, \mathsf{boolean}, \mathsf{integer}, \mathsf{number}, \mathsf{string}\}$, with the unconstrained formula acting as a sort-polymorphic constant, and a compound `And`/`Or` is well-formed only if all its non-unconstrained sub-formulae share one sort σ, which then becomes the compound's sort. Mixing sorts — `And(Animal, 3)`, combining a type constraint with an integer constant — is a static semantic error detected at constraint-construction time, before any instantiation is attempted.

The formulae of each sort, ordered by inclusion of their admitted sets, form a Boolean algebra in which `And`, `Or`, and `Not` are meet, join, and complement, with the unconstrained formula as its greatest element $\top$. Each sort carries its own least element $\bot_\sigma$, which has no constraint syntax of its own and arises only from algebraic simplification: `"Not(Concept)"` is $\bot_{\mathsf{type}}$, capturing that no admissible *Type* lies outside the Concept Hierarchy, and dually `Not(Literal:`σ`)` is $\bot_\sigma$, since no literal lies outside its declared domain. $\bot$ is indexed by sort while $\top$ is not; that is forced rather than notational: a single global $\bot$, carrying no sort marker, would leave `"Not(⊥)"` without a recoverable sort, so `"Not(Not(Concept))"` would collapse to a $\top$ admitting `3.1415` or `true` rather than back to the *Types* `"Concept"` admits.

Write $\mathit{admitted}(F)$ for the set of *Types* or literal values that a formula F admits, and let $F \models G$ hold exactly when $\mathit{admitted}(F) \subseteq \mathit{admitted}(G)$. This entailment order is the inclusion order of the Boolean algebra just described, so $\top$ is its greatest and $\bot_\sigma$ its least element, and it is decided without search: for the type sort, $\mathit{admitted}(F)$ is fixed by the five hierarchy operators of subsubsection 5.1.3, and $\models$ reduces to subsumption and disjointness lookups over the Concept Hierarchy; for the literal sorts, it reduces to containment of a value in its declared domain.

5.1.5 Constraints On Parameterized Concepts

When the concept T of a hierarchy operator is itself a parameterized *ValueDomain*, the constraint may optionally qualify the operator with sub-constraints on T's template arguments: T<F_1, ..., F_k>, where each F_i is a `templateConstraintFormula`, giving the constraint language its recursive structure. The number k of supplied sub-formulae must equal T's declared template parameter count (a variadic parameter counts as one); a mismatch is a static semantic error. The bare reference T and the empty-bracket form T<> are semantically distinct: the former places no restriction on T's own template arguments, admitting any type application of T; the latter is valid only if T has exactly one template parameter, whose constraint is then set to unconstrained.

When sub-constraints are present, checking whether a candidate *Type* C satisfies T<F_1, ..., F_k> proceeds in two steps. First, the hierarchy operator constraint is checked for C and T. Second, if the hierarchy operator constraint is satisfied, there are two cases.

If $C \sqsubseteq T$, the subtype declaration of C under T induces a substitution assigning a value to each of T's template parameters: the values with which C's declaration specializes T. Each F_i is then evaluated against the value that substitution assigns to T's i-th template parameter. The sub-constraints do not restrict C's template arguments, but the arguments C's subtype declaration supplies to T.

If $T \sqsubset C$, the performed check is whether there is a type application of T, T<v_1, ..., v_k>, that satisfies its defined sub-constraints, $\forall i : v_i \in \mathit{admitted}(F_i)$, and is a subtype of C: $T\langle v_1, \ldots, v_k\rangle \sqsubseteq_T C$.

5.1.6 Template Variables In Constraints

A hierarchy operator may name a *template variable* — a template parameter of the *ValueDomain* defining the constraint — in place of a concept reference T, letting the admissible set of one parameter depend on the value chosen for another, a cross-parameter dependency no per-slot constraint can capture. Because the type application of the referenced variable is unknown at definition time, such a reference may not carry sub-constraints in angle brackets; doing so is a static semantic error.

Consider a `DifferentPair` *ValueDomain* declaring `X` and `Y` with constraints `"ValueDomain"` and `"And(ValueDomain, Not(X.))"` respectively. `X` admits any *Type* descendant of *ValueDomain*, while `Y`'s conjunction admits any such *Type except* whichever is bound to `X` at the point of type application. `DifferentPair<Number, String>` is therefore admissible, and `DifferentPair<Number, Number>` is rejected — and, because `Y`'s constraint mentions `X`, it is verifiable only once an argument for `X` is known, never independently of an enclosing type application.

5.1.7 Declaring Template Parameters And Their Constraints

Template parameters are declared in the **`order`** array of a *ValueDomain*'s **`templateContext`** `JSON object`; the declared order is significant, since a type application associates arguments with parameters positionally (subsection 5.2). A template parameter name is an uppercase-starting identifier, distinct from every other name in the same **`order`** array and from the name of any concept defined in the Concept Hierarchy; two different *ValueDomains* may, however, each declare a parameter of the same name. Its constraint is declared as a key–value pair `"<templateParamName>": "<templateConstraintFormula>"` directly inside **`templateContext`**, alongside **`order`** itself, with no risk of collision against the other **`templateContext`** keywords (**`order`**, **`variadicGroupIdentifiers`**, **`substitution`**), which are lowercase-starting.

A parameter listed in **`order`** without an explicit constraint defaults to `"ValueDomain"`, admitting every *Type* descending from `ValueDomain`, abstract ones included. This is deliberately narrower than `"Concept"` and reflects that most typed template parameters range over *ValueDomains* rather than *DomainConcepts*.[11] We choose it over the non-abstract form `"ValueDomain*"` because a type parameter's constraint states a lower bound on the structure an argument must offer, not a requirement that the argument itself be directly instantiable: a bound of `Numeric` is meant to admit its concrete descendants (`Integer`, `Real`, `Complex`, ...) without requiring `Numeric` to be instantiable. The starred form remains available wherever a *ValueDomain*'s semantics truly requires abstract *Types* to be excluded.

When a *ValueDomain* defines neither a **`substitution`** nor variadic group identifiers (subsection 5.2), and every template parameter takes the default constraint, **`templateContext`** may be written in shorthand: a bare `JSON`

[11] Among the *ValueDomains* defined for validating the Concept Hierarchy, only `Instance`'s template parameters range over *DomainConcepts* exclusively, via the constraint `"And(Concept, Not(ValueDomain))"`, as presented in section 6.

`array` of parameter names, interpreted directly as **`order`**. Listing 9 illustrates both this shorthand `JSON array` form and the general, canonical `JSON object` form.

5.2 Type Application Syntax

A type application must supply exactly one template argument for every template parameter declared in the applied *ValueDomain*'s **`templateContext`**. Non-variadic parameters are straightforward in this respect: each contributes exactly one argument. Variadic parameters, marked with the `...` suffix in **`order`**, are not: a variadic parameter's application is a *sequence* of zero or more arguments, all constrained by the same formula.

Definition 5.7 (Variadic Template Parameter)**.** A template parameter (definition 5.5) may be declared *variadic*, in which case it corresponds to zero or more template arguments in a type application rather than exactly one. Every argument supplied for a variadic template parameter is subject to the same constraint and the same type/literal classification as the parameter itself, since a variadic parameter has a single declared constraint shared by all supplied arguments.

Definition 5.8 (Variadic Group)**.** Let p be a variadic template parameter of a *ValueDomain* V. A *variadic group* for p in a type application of V is the (possibly empty) sequence of template arguments supplied for p, each of which must independently satisfy p's constraint formula.

A template argument need not be ground. Wherever a type application occurs inside the definition of a *ValueDomain* W — nested within W's **`instantiation`**, or in a **`substitution`** into a parent — W's own template parameters are in scope and may be used as arguments.

Definition 5.9 (Template Variable Reference)**.** Let W be a *ValueDomain* with template parameter t declared in its **`templateContext`**. A *template variable reference* to t is an occurrence of t's name as a template argument within a type application appearing in the definition of W (or of a *ValueDomain* nested in W's scope), standing for whatever *Type* or literal is ultimately supplied for t at the point W itself is applied.

We do not require variadic parameters to sit at the end of **`order`**; they may appear at any position. A type application may therefore mix non-variadic arguments, template variable references, and one or more variadic groups, which leaves the grammar an inherent ambiguity to resolve: given a flat, comma-separated argument list, it is not in general possible to tell which arguments belong to which variadic group, or which are non-variadic, without additional markup. The Concept Hierarchy resolves this with two mutually exclusive syntaxes for an argument list — the two alternatives of the `templateInstantiation` production in Listing 11.

```
TypeAppliedValueDomain  ::= ws '"' namedType '"' ws
// == Top-level named type ================================================
// Literals, variadicId, and '...' are all forbidden here. Lowercase names are allowed syntactically; semantic checks validate the names.
namedType               ::= name typeExtra
name                    ::= nameStartingWithLetter
typeExtra               ::= templateInstantiation?
templateInstantiation   ::= '<' (templateArgListWithGrp | templateArgListWithVar)? '>'

// == Template argument lists (two mutually exclusive modes) ==============
// A given '<...>' uses one mode or the other; mixing is a semantic error.
// Mode 1 - group mode: variadic groups allowed, variadicId forbidden
templateArgListWithGrp  ::= templateArgWithGrp (', ' templateArgWithGrp)*
templateArgWithGrp      ::= variadicGroup | literalValue | namedType

// Mode 2 - variadicId mode: variadicId allowed, variadic groups forbidden
templateArgListWithVar  ::= templateArgWithVar (', ' templateArgWithVar)*
templateArgWithVar      ::= variadicId (literalValue | (name ('...' | typeExtra)))
variadicId              ::= [!$]*

// == Variadic group ======================================================
// Only usable as a template argument in group mode. Elements may be literals or expandable names; variadicId is forbidden here.
variadicGroup           ::= '[' groupElementList? ']'
groupElementList        ::= groupElement (', ' groupElement)*
groupElement            ::= literalValue | (name ('...' | typeExtra))

// == Literals ============================================================
// Allowed only as template argument values and variadic group elements. May not be followed by '...'.
literalValue            ::= boolean | number | '\\"' character* '\\"'
```

Listing 11: The `EBNF` grammar for the definition of a template parameter substitution value.

5.2.1 Canonical Form

In the *canonical form* (`templateArgListWithGrp`), every variadic group is written explicitly as a bracketed list `[...]` (`variadicGroup`) with its elements separated by `', '`, while non-variadic arguments — named types, literal values, or template variable references alike — are written directly, without brackets, in their declared position. The form is unambiguous by construction: the grouping of arguments into variadic groups is given syntactically by the bracket structure, independent of any parameter metadata. A variadic slot may additionally be filled without brackets at all, but only to *forward* a variadic template variable reference as-is, written in its non-expanded form (definition 5.10, item 3). A non-variadic argument may freely appear as an element of a variadic group, alongside expanded variadic references (`T...`) and literals. Whether the position and repetition of elements in a variadic group matters depends on the semantics of the *ValueDomain*; there is no syntactic marker distinguishing these aspects of a variadic template parameter.

5.2.2 Shorthand Form

The *shorthand form* (`templateArgListWithVar`) instead prefixes each argument belonging to a variadic group with that group's *variadic group identifier*, a string over the two-character alphabet {`!`, `$`} assigned to the group in the applied *ValueDomain*'s **`variadicGroupIdentifiers`** field; non-variadic arguments carry no prefix. Because distinct groups are assigned pairwise distinct identifiers, the identifier accompanying an argument unambiguously determines its group: parsing recovers the canonical grouping by collecting all arguments sharing an identifier, in order of appearance, and the ordering significance noted above applies identically. The remaining, unprefixed arguments are matched in order against the *ValueDomain*'s non-variadic parameters, in their declared **`order`**, independently of where variadic groups are interleaved among them.

An identifier may prefix a literal value, a named type, or a non-variadic template variable reference, but never a variadic reference in non-expanded form — which has no valid position in the shorthand form at all, since non-expanded variadic references are not allowed inside a variadic group. A variadic reference must instead appear *expanded*, suffixed with `...` and prefixed with an identifier: writing `!T...` in the shorthand form is equivalent to placing `T...` inside the variadic group identified by `!` in the canonical form.

Definition 5.10 (Well-formed Type Application)**.** A type application is *well-formed* if it satisfies the following constraints:

1. **No mixing.** A single type application uses either the canonical form or the shorthand form for its entire argument list; a type application may not combine bracketed variadic groups with variadic-identifier-prefixed arguments.

2. **Valid variadic group identifiers.** The variadic group identifiers, i.e. the results of the `variadicId` production rule, must conform to the assignment fixed by the *ValueDomain*'s **`variadicGroupIdentifiers`** (subsubsection 5.2.3).

3. **Variadic slots need a group or a forwarded reference.** A variadic parameter's slot is filled either by a bracketed variadic group (possibly empty) or, exclusively in canonical form, by a bare, non-expanded template variable reference to a variadic parameter of the enclosing *ValueDomain*.

4. **No bare non-variadic value for a variadic slot.** A named type or a non-variadic template variable reference may not, by itself and unbracketed, fill a variadic parameter's slot.

5. **Expansion only inside a group, or as forwarding.** The expanded form of a variadic template variable reference (`T...`) is well-formed only as an element of a variadic group (canonical form) or, equivalently, as a variadic-group-identifier-prefixed argument (shorthand form).

6. **No non-expanded variadic reference in a variadic-group position.** A bare (non-expanded) reference to a variadic template parameter of the enclosing *ValueDomain* may not appear as an element of a variadic group in canonical form, nor may it be prefixed by a variadic group identifier in shorthand form. In both positions, it must instead appear in its expanded form (`T...`).

5.2.3 The `variadicGroupIdentifiers` Field

Declared alongside **`order`** in **`templateContext`**, this field maps each variadic template parameter's name to an identifier string matching `[!$]*`, the empty string being a legal match, and so governs which identifiers the shorthand form may use. Three requirements constrain the mapping. *Coverage*: either every variadic parameter is assigned an identifier or none is — a partial assignment is ill-formed, and an empty assignment simply makes the shorthand form unavailable for that *ValueDomain*, leaving only the canonical form. *Uniqueness*: all assigned identifiers are pairwise distinct, which the two-character alphabet never obstructs, since it admits arbitrarily long strings and so yields a fresh identifier for any number of variadic parameters. *Empty identifier restriction*: the empty identifier may be assigned to at most one variadic parameter (a consequence of uniqueness) and only if the *ValueDomain* declares no non-variadic parameters — which is what permits the fully unprefixed shorthand for a *ValueDomain* with a single variadic parameter and nothing to confuse its arguments with.

These are properties of the *ValueDomain*'s *definition*, checked once when the *ValueDomain* is defined, and are independent of, and prior to, the well-formedness of any type application using them.

5.3 Substitution of Parent Template Arguments

Declaring template parameters (subsubsection 5.1.7) determines the values over which a *ValueDomain* is parameterized; it does not determine how a type application of that *ValueDomain* relates to a type application of its parents. For a non-parameterized direct parent, this relation is trivial: the parent contributes a single, fixed type application, its own bare name. For a parameterized direct parent P, a type application of P is a family of *Types* indexed by P's template arguments, and a subconcept D of P must specify *which member of that family* it inherits from, possibly depending on D's own template parameters. The **`substitution`** `JSON object` of **`templateContext`** supplies this specification.

Definition 5.11 (Template Parameter Substitution)**.** Let D be a *ValueDomain* with template parameters Θ_D (possibly $\emptyset$), and let $P_1, \ldots, P_n$ be the *ValueDomains* declared as D's **`directParents`**. For every P_i and every $\theta \in \Theta_{P_i}$, D's **`substitution`** object associates θ with a *template parameter* ***`substitution`*** *expression* e_θ, generated by the grammar of Listing 12, whose free variables are from Θ_D, the template parameters of D. We write

$$\sigma_D \;=\; \{\, \theta \mapsto e_\theta \mid 1 \le i \le n,\ \theta \in \Theta_{P_i} \,\}$$

for the *template parameter substitution* of D. Parameters of distinct parents are formally distinct even when identically named, so σ_D is indexed by the disjoint union $\biguplus_i \Theta_{P_i}$, i.e. by qualified pairs (P_i, θ), rather than by parameter name alone. Subsubsection 5.3.2 presents the two ways of naming such a pair in D's **`substitution`** definition.

```
1  TemplateArgumentValueForSubstitution ::= LiteralValue | TypeAppliedValueDomain | (ws '[' (ws | (VariadicGroupValue (',' VariadicGroupValue)*)) ']' ws)
2  VariadicGroupValue                   ::= LiteralValue | TypeAppliedValueDomain | (ws upperCaseName '...' ws)
3  LiteralValue                         ::= ws (boolean | number | ('"\\"' character* '\\""')) ws
```

Listing 12: The `EBNF` grammar for the definition of a template parameter **`substitution`** expression. `TypeAppliedValueDomain` is defined in Listing 11.

5.3.1 Well-Formedness Of `substitution` Expressions

The expression e_θ must be syntactically well-formed with respect to θ:

- if θ is a non-variadic literal parameter, e_θ is a `LiteralValue` of θ's literal domain, or a reference to one of D's non-variadic literal template parameters constrained on that domain;
- if θ is a non-variadic type parameter, e_θ is a `TypeAppliedValueDomain`, possibly specifying template parameters of D, or a reference to a non-variadic type template parameter of D;
- if θ is variadic, e_θ is a reference to a variadic template parameter of D (denoting the expansion of D's variadic argument group in P_i's place), or a bracketed variadic group (definition 5.8): a `JSON array` whose elements are `LiteralValues`, `TypeAppliedValueDomains` (possibly referencing template parameters of D), non-variadic template parameters of D, or expanded variadic template parameters of D.

These restrictions follow from constraint satisfaction — a *ValueDomain* substituted for a literal template parameter could never satisfy that parameter's constraint — but are stated explicitly as a syntactic well-formedness condition on **substitution**, independent of any particular type application.

5.3.2 Substitution Identifier Syntax

The canonical entry `"<ParentConceptName>:<TemplateParameterName>"` identifies its qualified pair (P_i, θ) of σ_D. The shorthand `"<TemplateParameterName>"` is permitted only if no two direct parents of D declare a template parameter of that name; otherwise, both occurrences require the canonical form, since they denote distinct parameters that must individually receive (possibly unrelated) **substitution** expressions.

5.3.3 The *Type* Hierarchy Of A Concept Hierarchy

Definition 5.4 characterizes the *Type* hierarchy as the parent relation over *Types*, in which ground type applications are ordered by **directParents** and template substitution; σ_D is precisely what the latter qualification refers to.

Definition 5.12 (*Type* Hierarchy)**.** The *direct-type-parent relation* $\lhd$ relates a *Type*, not a concept, to each of its direct parents. For *DomainConcepts*, $C \lhd C'$ for every C' declared under **directParents**. For a ground type application $D{<}a_1, \ldots, a_k{>}$ of a *ValueDomain* D and every direct parent P_i,

$$D{<}a_1, \ldots, a_k{>} \lhd \begin{cases} P_i & \text{if } \Theta_{P_i} = \emptyset, \\ P_i{<}e'_{\theta_1}, \ldots, e'_{\theta_m}{>}, \text{ where } m = |\Theta_{P_i}| \text{ and } \forall \theta_j \in \Theta_{P_i} : e'_{\theta_j} = e_{\theta_j}[a_1, \ldots, a_k] & \text{if } \Theta_{P_i} \neq \emptyset, \end{cases}$$

where $e_{\theta_j}[a_1, \ldots, a_k]$ denotes replacing in e_{θ_j} every free occurrence of D's template variables $\in \Theta_D$ by the corresponding ground argument $a_1, \ldots, a_k$. The *Type hierarchy* $\sqsubseteq_T$ contains the reflexive-transitive closure of $\lhd$ over the combined universe of *DomainConcepts* and ground *ValueDomain* type applications; its full definition is given in subsection 6.6.

5.3.4 Total Substitution Specification

e_θ must be defined for every $\theta \in \Theta_{P_i}$ of every parameterized direct parent P_i, independent of whether D itself is parameterized: without e_θ, the $\lhd$-edge from D to P_i in definition 5.12 would be undefined. Even a non-parameterized *ValueDomain* must therefore substitute into all template parameters of its parameterized parents, and, declaring no template parameters of its own, its substitution is necessarily ground.

5.3.5 Induced Constraints

A **substitution** expression does more than record an edge of the *Type* hierarchy. By using $\vartheta \in \Theta_D$ in a type application, e.g. providing the substitution value of a parent's template parameter θ, the type application *induces* the constraint of θ on ϑ, over and above whatever D declares for ϑ locally. This is not particular to **substitution**: a leaf occurrence of ϑ in a type application inside an **instantiation** schema constrains ϑ no less, since in either case ϑ is passed where the applied *ValueDomain* admits only what its own declaration permits. Any type application in D's definition may therefore narrow Θ_D.

Definition 5.13 (Defined, Induced, and Resolved Constraints)**.** Let ϑ be a template parameter of a *ValueDomain* D. Its *defined constraint* is the formula given for it directly in D's **templateContext** (subsubsection 5.1.7). An *induced constraint* on ϑ is one contributed by a leaf occurrence of ϑ in a type application within D's concept definition: a constraint that ϑ inherits from being substituted as some other *ValueDomain*'s argument, rather than from its own declaration. The *resolved constraint* of ϑ is the conjunction of its defined constraint with every induced constraint contributed to it.

The constraint induced on ϑ by an occurrence inside a type application of a *ValueDomain* P is the *resolved* constraint P places on the parameter at that position, so definition 5.13 is recursive, and the recursion need not be well-founded: two *ValueDomains* may each occur in the other's definition, and a *ValueDomain* may occur in its own. However, conjunction is idempotent, and both the number of template parameters of a Concept Hierarchy

and the number of distinct conjuncts contributable to any one of them are finite. The induction, therefore, has a least fixpoint, and the resolved constraint for every template parameter is well-defined regardless of recursion. It is computed by initializing every parameter's constraint to its defined constraint and repeating the induction step of definition 5.13 over every type application of the Concept Hierarchy until an iteration adds no conjunct to any parameter; idempotence makes the iteration monotone, and the two finiteness bounds make it terminate.

It is a parent's *resolved*, not defined, constraint for θ against which a descendant's substitution expression e_θ is validated, because a parent's defined constraint may omit what the parent itself inherited from further up. Validating against D's *direct* parents alone nonetheless suffices: a parent's resolved constraint already conjoins everything induced upon it along its own ancestry, so nothing is lost by checking only at that level.

Nothing beyond that definition-site check rests on a resolved constraint. The well-formedness of a *ground* type application is decided by recursive descent over its arguments against the *defined* constraints alone, consulting no resolved constraint at any point, and is therefore exact. A resolved constraint serves only as an early diagnostic, rejecting at a *ValueDomain*'s own definition a specialization that could never be validly grounded; a conjunct not recorded there delays such a rejection to the first ground type application rather than forgoing it.

The examples below show how substitution constraints induced from a grandparent shape what a *ValueDomain* further down the hierarchy may legally do, and how a well-formed specialization can be semantically empty.

```
{
  |...|,
  "NumericSubSet": { "data": { "templateContext": {
    "order": ["N"],
    "N": "Numeric"
  }}},
  "SubTypeSubSet": { "directParents": ["NumericSubSet"], "data": { "templateContext": {
    "order": ["T", "SubT"],
    "T": "ValueDomain",
    "SubT": "And(T, Not(T.))",
    "substitution": {
      "NumericSubSet:N": "T"
    }
  }}},
  "Interval": { "directParents": ["SubTypeSubSet"], "data": { "templateContext": {
    "order": ["R"],
    "R": "Real",
    "substitution": {
      "SubTypeSubSet:T": "R",
      "SubTypeSubSet:SubT": "R"
    }
  }}}
}
```

Listing 13: `SubTypeSubSet` inherits from `NumericSubSet`, inducing a `"Numeric"` constraint on its own `T` beyond what it declares locally. `Interval` substitutes both template parameters of `SubTypeSubSet` with the same `R`, which no ground argument satisfies.

`NumericSubSet<N>` constrains `N` to `"Numeric"`. `SubTypeSubSet<T, SubT>` declares `T` with the comparatively permissive local constraint `"ValueDomain"`, but substitutes `T` for `N` in its parent — so, by definition 5.13, `T`'s *resolved* constraint is the conjunction of `"ValueDomain"` and `"Numeric"`, i.e. effectively `"Numeric"` alone, even though nothing in `SubTypeSubSet`'s own declaration says so directly. `SubT`'s constraint, `"And(T, Not(T.))"`, restricts it to a strict subtype of `SubTypeSubSet`'s own `T`.

`Interval<R>` substitutes *both* `T` and `SubT` with its own `R`. The constraint induced from the `SubT` slot requires `R` to be a strict subtype of whatever is substituted for `T` — which is `R` itself. No ground argument satisfies "strict subtype of itself," so `Interval` admits no valid type application at all: the definition is syntactically well-formed but denotes the empty family, and is rejected as an impossible specialization. This is detected entirely from `SubTypeSubSet`'s own resolved constraints on `T` and `SubT`; `NumericSubSet` is never consulted directly.

```
{
  |...|,
```

```
  "StringSubSet": { "directParents": ["SubTypeSubSet"], "data": { "templateContext": {
    "order": ["S"],
    "S": "String",
    "substitution": {
      "SubTypeSubSet:T": "ValueDomain",
      "SubTypeSubSet:SubT": "S"
    }
  }}}
}
```

Listing 14: A substitution that satisfies its direct parent's *locally declared* constraint but violates the constraint that parent resolved after induction from further up the hierarchy.

`StringSubSet<S>` fixes *T* to the ground top constraint `"ValueDomain"` rather than substituting one of its own template parameters, and substitutes *SubT* with its own *S*, constrained to `"String"`. Checked against `SubTypeSubSet`'s *local* declarations alone, both slots look fine: `ValueDomain` trivially satisfies *T*'s local constraint `"ValueDomain"`, and any proper subtype of `ValueDomain` — `String` included — satisfies *SubT*'s local constraint. But *T*'s *resolved* constraint, per Listing 13, is `"Numeric"`, not merely `"ValueDomain"` — and the fixed ground argument `ValueDomain` does not satisfy `"Numeric"`. `StringSubSet` is therefore rejected too, but for a different reason than `Interval`: not an internal collision between two of the parent's template parameters, but a plain violation of a constraint induced two levels up, in `NumericSubSet`, and carried forward by `SubTypeSubSet` into what it resolves for *T*.

5.4 The `instantiation` Interface

The value of the **`instantiation`** field is a schema written in a dialect of `JSON Schema` Draft-07 [Wright et al.(2018), Wright and Andrews(2018)]. We reuse Draft-07 for the same reason it is reused throughout the `JSON` tooling ecosystem: its vocabulary of structural constraints (**`properties`**, **`items`**, **`enum`**, **`allOf`**/**`anyOf`**/**`oneOf`**, ...) already covers most of what is needed to describe the *shape* of a `JSON` value. What it cannot express is that a schema leaf stands for an expression of a *ValueDomain* type application rather than a primitive `JSON` value; whether that expression must be a variable, with a fixed place in memory; that a set of object keys is drawn from a *DomainConcept*'s declared **`properties`** or **`functions`** rather than listed literally; that the schema's shape may itself depend on a type application's template arguments; and that a `JSON string` value or object key must name a concept or *Type* of the Concept Hierarchy. Our dialect extends Draft-07 to cover these gaps, as summarized below and detailed in the remainder of this subsubsection.

- **Shorthands.** A bare type name or a two-element array may stand in place of a full schema object (subsubsection 5.4.1).
- **Provenance character.** The new keyword **`provenance`** records whether a Concept Hierarchy *Type* value must be addressable (**`"Addr"`**) or not, i.e. **`"Any"`**, a distinction Draft-07 has no notion of (subsubsection 5.4.2).
- **Concept and *Type* name constraints.** A `JSON string` leaf (incl. `JSON object` keys) may be constrained by **`"format"`**: `"Concept"` or **`"format"`**: `"Type"`, with an optional companion **`constraint`** formula, to admit only the names of concepts or *Types* from a restricted set (subsubsection 5.4.3).
- **Opaque Concept Hierarchy *Type* leaves.** A schema object whose **`type`** is a Concept Hierarchy *Type* may carry no further structural keywords: its instantiation structure is defined by that *ValueDomain*'s own **`instantiation`** schema, not by nesting Draft-07 keywords around it (subsubsection 5.4.4).
- **Literal template parameters as constraint values.** Wherever Draft-07 expects a boolean, integer, numeric, or string literal (e.g. **`minimum`**, **`maxItems`**, ...), our dialect also accepts the name of a literal template parameter of the enclosing *ValueDomain* (subsubsection 5.4.5).
- **Concept-data key constraints.** Complementary to the **`properties`** keyword in Draft-07, which specifies a `JSON` object of literal key names, the custom **`properties`** keyword may list one or more *concept-data key specifications* drawing keys from a *DomainConcept*'s declared or inherited properties/functions (subsubsection 5.4.6).

- **Template-dependent instantiation.** The `instantiation` value may itself be a list of (template-argument constraints, schema) pairs, letting a parameterized *ValueDomain* choose a different instantiation shape for different template arguments (subsubsection 5.4.8).
- **Schema reuse.** `$ref` is extended with an `#ch#/`⟨*conceptName*⟩ prefix for referencing definitions inside another *ValueDomain*'s `instantiation` schema (subsubsection 5.4.9).

5.4.1 Shorthands

Because most leaves of an `instantiation` schema constrain a single type with no further refinement, two shorthands avoid the boilerplate of a full schema object. A bare `JSON string` s used where a schema is expected expands to `{"type": s}` if s names one of the seven Draft-07 primitive types, and to `{"type": s, "provenance": "Any"}` if s is instead a type application (definition 5.2) of a *ValueDomain* – a *custom Type* – with no `default` value. A two-element `JSON` array `[s, r]`, with s a *Type* name and $r \in$ {`"Addr"`, `"Any"`}, expands to `{"type": s, "provenance": r}`; using this array form with a builtin primitive type name is a syntax error, since a provenance character is only meaningful for Concept Hierarchy *Types*. Both shorthands are unambiguous extensions of Draft-07: a bare `JSON string` is never itself a valid Draft-07 schema, and neither is a bare `JSON array`, so no existing Draft-07 meaning is displaced.

```
{ "type": "object", "properties": {
  "position": "Point3D",              // Concept Hierarchy Type, any provenance (Any, the default)
  "frame":    ["Frame", "Addr"],      // Concept Hierarchy Type, addressable values/expressions only
  "label":    "string"                // JSON (Schema) builtin type
}}
```

Listing 15: Shorthand instantiation schemas. A bare *Type* name defaults the provenance character to `Any`, admitting an addressable value and an inline-created one alike; the array form may restrict a leaf to `Addr`.

5.4.2 The *Addr*/*Any* Distinction

Definition 5.14 (Provenance Character)**.** Every Concept Hierarchy *Type* leaf of an `instantiation` schema carries a *provenance character* *provenance* $\in$ {`"Addr"`, `"Any"`}, given explicitly via the `provenance` keyword or defaulted by the shorthands of subsubsection 5.4.1. Informally, the relationship between `Addr` and `Any` mirrors that between an l-value and an r-value in `C++`: one cannot bind the l-value slot expected by an assignment to the literal r-value `2`. Similarly, an `Addr` leaf does not accept a value created inline – the value must instead come from something addressable, i.e. already given a location, such as a variable, an instance property, or the result of a *Function* evaluation whose return *Type* has an `Addr` character.

5.4.3 Constraining A `JSON` *string* To A *Concept* Or *Type* Name

A `JSON string` leaf of an `instantiation` schema often stands not for arbitrary text but for the *name* of a concept or a *Type* drawn from the Concept Hierarchy— a single-key *Function* composition `JSON object`, for instance, is keyed by the name of the *Function* it evaluates. Draft-07's `format` keyword is the natural place to record such a restriction: it is defined precisely as a vocabulary of named, string-valued assertions layered on `{"type": "string"}` [Wright et al.(2018)], and its registry is entirely lowercase (`"date-time"`, `"uri"`, `"regex"`, ...), so the two capitalized values our dialect introduces, `"format": "Concept"` and `"format": "Type"`, cannot collide with any standard format. The former restricts the `JSON string` to the name of a *DomainConcept* or *ValueDomain*; the latter, to a *Type*, i.e. a ground type application (definition 5.4).

A bare `"Concept"` or `"Type"` format admits any concept or *Type* name, respectively. To narrow the admitted set, the leaf may carry a companion `constraint` keyword whose value is a constraint formula in the language of subsection 5.1. For `"Type"`, the `constraint` is a type-sorted `templateConstraintFormula` in full: all five hierarchy operators (subsubsection 5.1.3), the `And`/`Or`/`Not` connectives (subsubsection 5.1.4), and the recursive template-argument sub-constraints of subsubsection 5.1.5. The `"Concept"` constraint uses the identical formula language save for one omission: because a concept names a *family* of *Types* rather than a *Type* — `List`, not `List<String>` — an atom in a `"Concept"` constraint may not carry the `<...>` template-argument qualification

that would restrict that family to particular applications. Listing 16 gives the only two different productions from Listing 10.

```
conceptConstraintFormula
        ::= (('And' | 'Or') '(' conceptConstraintFormula (', ' conceptConstraintFormula)* ')') | ('Not(' conceptConstraintFormula ')') | conceptConstraint
conceptConstraint
        ::= (upperCaseName '*'?) | ('^' upperCaseName '*'?) |  (upperCaseName '.')
```

Listing 16: The *Concept*-constraint grammar as a delta over Listing 10: `conceptConstraint` replaces `typeTemplateConstraint`, and its atoms are bare `upperCaseName`s with no `<...>` qualification (nor the `literalTemplateConstraint` alternative, the type sort being the only admissible one). All other productions — `And` / `Or` / `Not`, `upperCaseName` — are as in Listing 10.

Both **`format`**s apply wherever a `JSON string` schema is admissible, including in the **`propertyNames`** schema of a `JSON object`[12]. Thus, every `string` value, including `JSON object` keys, may be constrained to concept or *Type* names.

5.4.4 Opaque Concept Hierarchy *Type* Leaves

A schema object whose **`type`** is a Concept Hierarchy *Type* is a *leaf* with respect to the surrounding Draft-07 structure: it may specify only **`type`**, **`provenance`**, **`default`**, and the purely descriptive **`title`**/**`description`**/**`$comment`**; every other Draft-07 keyword (**`properties`**, **`items`**, **`enum`**, . . .) is syntactically incorrect at this position. This reflects the opacity described at the start of this subsection: once a leaf names a *Type*, how a `JSON` value is recognized and validated as an instantiation of it is entirely the concern of *that Type*'s own **`instantiation`** schema, not of the schema that references it. The **`default`** keyword also changes its meaning at such a leaf: in Draft-07, **`default`** is a non-normative annotation with no validation [Wright et al.(2018)], whereas here, when the surrounding `JSON` value omits a value for this position, the recorded **`default`** is used as the instantiating expression instead, and is independently validated as an instantiation of the leaf's *Type*.

5.4.5 Literal Template Parameters As Constraint Values

A parameterized *ValueDomain* (definition 5.1) may declare a non-type, i.e. *literal*, template parameter (definition 5.5) and use it inside its own **`instantiation`** schema. Wherever Draft-07 requires a boolean, integer, number, or string value, for example **`minimum`**, **`maximum`**, **`maxLength`**, or **`minLength`**, our dialect additionally accepts a string value naming such a literal template parameter as a placeholder. The placeholder is substituted during monomorphization (definition 5.4), at which point the surrounding schema object reduces to an ordinary Draft-07 constraint and is validated as such; a **`instantiation`** schema containing unsubstituted placeholders is therefore only well-formed relative to a template context that leaves the corresponding parameter free.

5.4.6 Concept-Data Key Constraints

The remaining extension addresses `JSON object` schemas whose keys are not known in advance, because they are drawn from a *DomainConcept*'s own declared vocabulary of properties or functions.

Definition 5.15 (Concept-Data Key Specification)**.** A *concept-data key specification* is a triple (κ, σ, ρ) where:

- κ is one of the six forms in Table 8, naming a set of *DomainConcept* properties or functions;
- σ is a `JSON Schema` shorthand definition (string or array) in which the type can specify the template parameter x, which represents the respective *Type* of the property or the function matched by κ; and
- ρ is an optional boolean (default `false`): if `true`, every key matched by κ must be present in the instance.

Instead of an `object` of literal key names, the **`properties`** keyword may hold a single concept-data key specification or a `JSON` array of several. The two forms are mutually exclusive with the ordinary Draft-07 `object` form, and with each other, so a single schema object either lists literal keys or draws keys from concept data, never

[12] Recall that **`propertyNames`** constrains an `JSON object`'s *keys*, each of which is necessarily a `JSON string`; a `"Type"`-formatted **`propertyNames`** schema thus requires every key to name a *Type*. The **`instantiation`** schema of `FunctionComposition` (subsubsection 7.3.4) relies on this.

Table 8: The six concept-data key forms, $C_1, \ldots, C_n$ ranging over *DomainConcept* names; $n \geq 1$.

Form	Matched keys
`props`	every declared property of every *DomainConcept*
`funcs`	every declared function of every *DomainConcept*
`props`$(C_1, \ldots, C_n)$	properties declared directly in $C_1, \ldots, C_n$ (not inherited)
`funcs`$(C_1, \ldots, C_n)$	functions declared directly in $C_1, \ldots, C_n$ (not inherited)
`props+`$(C_1, \ldots, C_n)$	properties declared in or inherited by $C_1, \ldots, C_n$
`funcs+`$(C_1, \ldots, C_n)$	functions declared in or inherited by $C_1, \ldots, C_n$

both at the same nesting level. When several specifications are listed together, the object's admitted keys are their *union*: each specification independently contributes its matched keys, its schema σ, and its requiredness ρ. `additionalProperties` may be specified to either restrict or allow other keys in the `object` to be matched.

A specification is written as a `JSON` array of its components in the order of definition 5.15: `[`κ`, `σ`]` or `[`κ`, `σ`, `ρ`]`, the two-element form leaving ρ at its default. Because a specification is itself an `JSON array`, the single and the list forms are told apart by the `JSON` type of the first element: a `JSON string` means `"properties"` holds one specification, an `JSON array` that it holds a list of them.

Because σ constrains the *value* found at each matched key, and that value's *Type* is already fixed by whatever the *DomainConcept* declared for that property or function, σ is typically a reference back to it or a type application using it. This is the purpose of the reserved lowercase template-parameter-like `x`[13]: written as the schema of a concept-data key specification, `x` stands for "whatever *Type* this *DomainConcept* already declared at this key," letting the specification add only a provenance character, `["x", "Addr"]`.

5.4.7 Expansion of Variadic Template Arguments

A schema may reference a variadic template parameter (definition 5.7) wherever it may reference any other, but a variadic group (definition 5.8) binds zero or more arguments to the single occurrence the schema shows. The expansion operator `...`, suffixed to the parameter's name, resolves this: an occurrence of `P...` stands for the whole group bound to `P` by the enclosing type application, and monomorphization replaces the occurrence with one copy per argument. What is copied is not the occurrence alone but the schema fragment around it, and how much of that fragment is fixed by where an enumeration of copies can be written at all.

Definition 5.16 (Variadic expansion)**.** Let a schema S contain an occurrence of `P...`, and let μ be the innermost `JSON Schema` keyword enclosing that occurrence whose value may be a `JSON` array, e.g. `"anyOf"`, `"items"`, or `"enum"`, or an enumeration in the custom syntax, e.g. inside the `"props(...)"` concept list, found by ascending from the occurrence. Write μ's value as $[\ldots, e, \ldots]$, with e the element containing the occurrence and a non-array value v read as the singleton $[v]$. Under a type application binding `P`'s variadic group to $(p_1, \ldots, p_m)$, monomorphizing S replaces e by the m elements $e[\texttt{P...} \mapsto p_1], \ldots, e[\texttt{P...} \mapsto p_m]$, in the group's order. An occurrence of `P...` with no such enclosing keyword or an occurrence of a variadic template parameter without the expansion operator `...` makes S ill-formed.

Expansion is thus outward-bound but minimal: it climbs from the occurrence only as far as the nearest position that admits a list, and no further. Under a type application binding `P` to `["String", "Number", "List<Vector<3>>"]`, the schema `{ "anyOf": ["P..."] }` monomorphizes to `{ "anyOf": ["String", "Number", "List<Vector<3>>"] }`, a value of any one of the three *Types*.

5.4.8 Template-Dependent Instantiation

A parameterized *ValueDomain*'s natural `JSON` encoding may itself depend on which template arguments it is applied to: `Map<K, V>`, say, might instantiate as an `array` of two-element key-value `arrays` in general, but as a `JSON object` with `string` keys specifically when `K` is applied to `String`, see Listing 17.

[13] `x` is not a declared template parameter, and thus does not follow uppercase-initial convention of Table 1. It is a reserved placeholder, admissible only as the σ of a concept-data key specification, and it is never declared, never supplied an argument, and never referable elsewhere. It nonetheless occupies a type-application position, which is why the grammar of Listing 11 admits any letter-initial identifier there; a lowercase identifier other than `x` is rejected by a semantic check rather than by the grammar.

Definition 5.17 (Template-Dependent Instantiation)**.** The **instantiation** value of a parameterized *ValueDomain* may be a nonempty list of pairs $(\vec{F}, \sigma)$ in place of a single schema, where σ is `JSON Schema` of our dialect and $\vec{F} = (F_1, \ldots, F_k)$ is a k-tuple of template argument constraint formulae, one per each template parameter declared in the *ValueDomain*'s **templateContext order** (Listing 9).

```
"Map": { "directParents": ["ValueDomain"], "data": {
  "templateContext": ["K", "V"], "instantiation": [
    [["String", ""], { "type": "object", "additionalProperties": "V" }],
    [["", ""], {
      "type": "array",
      "items": { "type": "array", "items": ["K", "V"], "minItems": 2, "maxItems": 2 }
    }]
] } }
```

Listing 17: Template dependent **instantiation** schema for the `Map` *ValueDomain*.

The order of the instantiation definitions determines which `JSON Schema` an instantiation of a type application must satisfy: the first pair whose constraint tuple the type application's template arguments satisfy is selected. Well-formedness of a template-dependent **instantiation** requires a *fallback* pair whose constraint tuple is entirely unconstrained, $\vec{F} = (\texttt{""}, \ldots, \texttt{""})$, listed last so that no pair after it becomes unreachable, guaranteeing that **instantiation** remains defined however the *ValueDomain* is used.

A non-abstract *ValueDomain* that declares no **instantiation** at all defaults to the schema of the empty `JSON` object, `{"type": "object", "maxProperties": 0}`; an abstract *ValueDomain* may not declare an **instantiation** schema at all, since it can never be instantiated and the **instantiation** is not inherited.

5.4.9 Schema Reuse

As in Draft-07, **$ref** may point at any depth into the current document via a `JSON` Pointer (e.g. `"#/$defs/Point3D/properti` or at a schema in a remote document identified by URI, whether hosted on the local file system or elsewhere on the internet.

Our dialect extends **$ref** with one Concept Hierarchy-specific target. Because every **instantiation** schema is itself nested inside a *ValueDomain*'s definition within the Concept Hierarchy, a reference of the form

$$\texttt{"\#ch\#/}\langle \textit{conceptName} \rangle \texttt{[/}\langle \textit{index} \rangle \texttt{]/}\langle \textit{pointer} \rangle \texttt{"}$$

addresses a definition inside *another* *ValueDomain* or *DomainConcept*'s own **instantiation** schema, named by $\langle \textit{conceptName} \rangle$, as though that schema were a separate document: the `#ch#/`$\langle \textit{conceptName} \rangle$ prefix selects the target document, and the remaining $\langle \textit{pointer} \rangle$ – itself an ordinary local `JSON` Pointer of the form `#/$defs/`$\langle \textit{name} \rangle$, at any depth – is resolved within it. When the target's **instantiation** is given in the template-dependent form of subsubsection 5.4.8, it is a list of pairs rather than a single schema with its own **$defs** to resolve $\langle \textit{pointer} \rangle$ against; the optional 0-based $\langle \textit{index} \rangle$ segment then selects which pair's schema σ the reference resolves into (e.g. `"#ch#/Map/1/#/$defs/Entry"` addresses the second pair of `Map`'s **instantiation** list). $\langle \textit{index} \rangle$ and $\langle \textit{pointer} \rangle$ are never ambiguous: $\langle \textit{pointer} \rangle$ always begins with `#/`, whereas $\langle \textit{index} \rangle$ is a bare non-negative integer, so a reference either omits the index (target is a single schema) or supplies exactly one before the pointer (target is template-dependent). This lets a family of related *ValueDomains* share substructure, e.g. a common coordinate encoding, by defining it once and referencing it in every **instantiation** schema that needs it, rather than duplicating it.

```
"Path3D": { "directParents": ["ValueDomain"], "data": { "instantiation": {
  "type": "object",
  "properties": {
    "start": { "$ref": "#ch#/Point3D/#/$defs/Coordinate" },
    "end":   { "$ref": "#ch#/Point3D/#/$defs/Coordinate" }
}}}}
```

Listing 18: Referencing a definition from another *ValueDomain*'s **instantiation** schema.

5.4.10 Instantiation And Subsumption

The defined subtype relation $\sqsubseteq_T$ is *declared* and not read off the `"instantiation"` schemas: it is fixed by `"directParents"` and by template substitution and, for the reifying family, by Definitions 6.5 and 6.6. An `"instantiation"` schema is not inherited (subsubsection 5.4.8), and, thus, a subtype's schema is not obliged to admit a subset of the `JSON` values its supertype's schema admits.

A schema fixes the syntax that *generates* a *ValueDomain*'s values: two *ValueDomains* may generate the same value from different `JSON` values, or two *ValueDomains* may generate different values from the same `JSON` value. Thus, a subtype narrowing what its denoted values *are* may widen how the values may be *written*. Subsumption between *ValueDomains* is a statement about the sets of values two *Types* denote, and not about the two `instantiation` schemas as syntax.

5.5 The `defaultSerialization` Mechanism

While `instantiation` (subsection 5.4) recognizes a *ValueDomain* against an already-known expected *Type*, a bare `JSON` value with no expected *Type* – e.g. a numeric literal where several *ValueDomains* could apply – has no `instantiation` schema to match against yet. `defaultSerialization` resolves this case by associating a *ValueDomain* with one `JSON` primitive type, so that a bare value of that type is interpreted as an instantiation of that *ValueDomain* when no expected *Type* is available (section 8).

Definition 5.18 (Default Serialization)**.** A *default serialization* $\tau \in \{$`"null"`, `"boolean"`, `"integer"`, `"number"`, `"string"`$\}$ is a `JSON` primitive type associated with a *ValueDomain* via its `defaultSerialization` field; τ must be unique among all *ValueDomains*, so that a bare value of that type has a unique interpretation.

A *ValueDomain* may leave `defaultSerialization` undefined. Like the `instantiation` keyword, it is not inherited by subconcepts, which is what keeps the uniqueness of definition 5.18 satisfiable. Three kinds of *ValueDomain* may not define one at all: parameterized *ValueDomains*, since template arguments generally cannot be inferred from a bare value; abstract *ValueDomains*, which – as with `instantiation` – are never themselves instantiated; and *Functions*, because the evaluation syntax recognizes them, not their instantiation schema. The composite types `"array"` and `"object"` are excluded from τ's range, as interpreting either may require structural validation unavailable without an expected *Type*. The bare `JSON` literal `null` and the `JSON string` `"null"` play different roles here: `defaultSerialization: "null"` declares $\tau =$ `"null"`, i.e. that the literal `null` instantiates this *ValueDomain*; `defaultSerialization: null` leaves τ undefined.

5.6 `Variation`: Customizable Subsets of a *Type*

A *Type* denotes a set of values (definition 5.4); the `Variation<T>` *ValueDomain* denotes a *subset* of the set denoted by `T`. This is the implicit-definition discipline of this subsection, applied to a *ValueDomain*'s own extension: where `T` fixes which values exist, a `Variation<T>` fixes which of them are admitted. The subsets so denotable range from the two trivial ones — the empty set, and all of `T`'s values — through a single value, to any logical combination[14] of other `Variation<T>` values [Costinescu et al.(2025)].

The work is done by `Variation`'s subconcepts, each of which fixes a *structure* for the subsets it denotes and leaves that structure's parameters to be supplied when it is instantiated. Parameterization here is not template- or type-parameterization: it is the ordinary instantiation of a *ValueDomain* whose `instantiation` schema (subsection 5.4) exposes the arguments, i.e. parameters, of the subset it describes. `Interval<Numeric>`, a subtype of `Variation<Numeric>`, denotes the contiguous subsets of a numeric *Type*; *which* contiguous subset is customized at instantiation, by supplying `lowerBound` and `upperBound`. A `Variation` subconcept is, in this sense, an implicit and parameterized subset definition, just as a *ValueDomain* is an implicit definition of a set.

This is what makes `Variation` suitable for defining a property's `constraint` (definition 4.3). A `constraint` must carve a subset out of the property's `valueDomain T` without introducing a second, competing type discipline alongside the one this subsection defines. `Variation<T>` is precisely the *ValueDomain* whose instances *are* such subsets of `T` values.

[14] by defining the `VariationConjunction<T>`, `VariationDisjunction<T>`, and `VariationNegation<T>` subconcepts

6 Reifying *DomainConcepts* as *Types*

ValueDomains define the Concept Hierarchy's *Types* (definition 5.4) and *DomainConcepts* describe the application domain, but the two are not interchangeable: every site in a Concept Hierarchy at which a value is expected — a property's `"valueDomain"`, a *Function* argument, an argument of a `"instantiation"` schema, or a global variable — is typed by a *ValueDomain Type*, and never by a *DomainConcept* directly[15]. Both determine sets of values, but only one of the two ways of determining them makes a *Type*. A *ValueDomain* is generative: its `"instantiation"` schema (subsection 5.4) states the syntax that generates all its values. A *DomainConcept* is restrictive: it states requirements an instance must satisfy, and presupposes a universe of instances rather than generating it. A *Type* must be generative, because a set whose members cannot be written is of no use as the *Type* of a property or a variable: no expression could denote one. Generativity is the only thing a *DomainConcept* lacks — its requirements are already exactly those a *Type* would impose — and this subsection supplies it, so that those requirements can be imposed at the site where a *Type* is needed while the *DomainConcept* itself remains stated once under `"concepts"`.

Definition 6.1 (Reification of a *DomainConcept*)**.** A *Type* τ *reifies* a *DomainConcept* C if τ may be written wherever a *Type* is expected and admits exactly the values that satisfy C's requirements — omitting no requirement of C and adding none of its own. Reification is thus just a change of syntactic category: C remains defined solely by its *DomainConcept* definition under `"concepts"`, and τ refers to that definition rather than restating it.

Which values a *Type* admits is a statement about extensions, so definition 6.1 is discharged in subsection 6.5 and not here; the present section gives the syntax it speaks of. C is unaffected either way. Its subsumption, disjointness, instance-count bounds, and property definitions stay where they are, and the confidence a value carries attaches to the value an expression evaluates to, not to the *Type* at which the expression is written, so it too needs no place in the reification. Not every ground type application of the family below reifies a concept, and subsection 6.5 states what the others admit.

The family rests on auxiliary *ValueDomains*, presented bottom-up: `ConceptValue` and `TypeValue`, which name a concept and a type application respectively (subsection 6.1); `InstanceDataBase` and `InstanceData<AcceptConcepts...>`, containing an instance's property and function values (subsection 6.2); and `InstanceBase` with its parameterized subconcept `Instance<AcceptConcepts..., RejectConcepts...>`, which are the reification *ValueDomains* (subsection 6.3). Subsection 6.4 adds `Variant<T...>`, without which the family reaches only conjunctions of *DomainConcepts*; subsection 6.5 states which instances a ground application of the family admits, which applications reify a concept, and that the family reaches every Boolean combination of the Concept Hierarchy's *DomainConcepts*; and subsection 6.6 gives the subtype relation among the resulting type applications.

6.1 Naming Concepts and Type Applications: `ConceptValue` and `TypeValue`

A reified *DomainConcept* must be able to record, as data, which concept it instantiates, and this requires a *ValueDomain* whose values are concept names. The difficulty is that a bare `JSON string` is already an expression form: by section 8, a `JSON string` in a value position is read as a variable reference, so a concept name written plainly would be indistinguishable from a variable that happens to carry the same name. `ConceptValue` and `TypeValue` resolve this with a syntactic marker, shown in Listing 19.

```
{
  |...|,
  "ConceptValue": { "directParents": ["String"], "data": { "instantiation": {
    "type": "string", "pattern": "^s:", "format": "Concept"
  }}},
  "TypeValue": { "directParents": ["String"], "data": { "instantiation": {
    "type": "string", "pattern": "^s:", "format": "Type"
  }}}
}
```

Listing 19: The `ConceptValue` and `TypeValue` *ValueDomain* definitions. Both are subconcepts of `String` and both use the `"format"` keyword of subsubsection 5.4.3.

[15] In a Concept Hierarchy, a *DomainConcept Type* is not a usable type of a property for example; it is only a valid template argument.

The `"pattern"` keyword requires the `s:` prefix, which marks the `JSON string` as a literal value, not a variable in the current scope. The `"format"` keyword then fixes what kind of `JSON string` literal: `"Concept"` admits exactly the names of concepts of the current Concept Hierarchy, and `"Type"` the type applications over them (subsubsection 5.4.3). `TypeValue` is not required by the reification itself and is given here only because it is the companion of `ConceptValue` sharing the same `"format"` extension.

6.2 Property and Function Values: `InstanceDataBase` and `InstanceData<...>`

The second auxiliary is a *ValueDomain* holding the data an instance carries: the values of the properties and functions that its *DomainConcepts* define or inherit (section 4). This is exactly what the concept-data key constraints of subsubsection 5.4.6 express — a schema whose admissible object keys are the properties and functions of a given set of *DomainConcepts* — and Listing 20 applies them.

```
{
  |...|,
  "InstanceDataBase": { "directParents": ["ValueDomain"], "data": { "instantiation": "object" } },
  "InstanceData": { "directParents": ["InstanceDataBase"], "data": {
    "templateContext": {
      "order": ["AcceptConcepts..."],
      "AcceptConcepts": "And(Concept, Not(ValueDomain))",
      "variadicGroupIdentifiers": { "AcceptConcepts": "" }
    },
    "instantiation": {
      "type": "object",
      "properties": [["props+(AcceptConcepts...)", "x"], ["funcs+(AcceptConcepts...)", "x"]],
      "additionalProperties": true
    }
  }}
}
```

Listing 20: The `InstanceDataBase` and `InstanceData<AcceptConcepts...>` *ValueDomain* definitions. The concept-data constraint (subsubsection 5.4.6) of `"properties"` restricts the expected *Type* of the keys' values. Per the open-world assumption, additional keys in the `object` are allowed.

In accordance with the open-world assumption, `InstanceDataBase` does not restrict its content to defined properties or functions in a Concept Hierarchy. `InstanceData<AcceptConcepts...>` constrains the keys that are properties or functions of the concepts in `AcceptConcepts` to match their defined `valueDomain`, `x`; it does not restrict the keys to only the properties and functions of those concepts, nor does it require all those properties and functions to be defined[16]. Because property and function names are unique across an entire Concept Hierarchy (definition 4.2), a key alone determines the expected *Type*, `x`, of the expression bound to it.

6.3 The Reification: `InstanceBase` and `Instance`

`InstanceBase` is the *ValueDomain* that reifies all *DomainConcept* instances of the application domain. It is written as a *Type* where any instance is admissible. Its parameterized subconcept `Instance<AcceptConcepts..., RejectConcepts...>` narrows that set to the instances that satisfy the requirements of every concept listed in the variadic `AcceptConcepts` group and of none of the concepts in `RejectConcepts`. Listing 21 gives both definitions.

```
{
  |...|,
  "InstanceBase": { "directParents": ["ValueDomain"], "data": { "instantiation": {
    "type": "object",
    "properties": {
      "concepts": { "type": "List<ConceptValue>", "default": [] },
      "properties": "InstanceDataBase",
      "instanceName": { "type": "string", "pattern": "^s:[a-zA-Z][a-zA-Z0-9_]*$" }
```

[16] Which reading an omitted key receives — unknown, or a closed-world `"default"` — is fixed by the property's own definition (definition 4.4).

```
    }
  }}},
  "Instance": { "directParents": ["InstanceBase"], "data": {
    "templateContext": {
      "order": ["AcceptConcepts...", "RejectConcepts..."],
      "AcceptConcepts": "And(Concept, Not(ValueDomain))",
      "RejectConcepts": "And(Concept, Not(ValueDomain))",
      "variadicGroupIdentifiers": { "AcceptConcepts": "", "RejectConcepts": "!" }
    },
    "instantiation": {
      "type": "object",
      "properties": {
        "concepts": { "type": "List<ConceptValue>", "default": [] },
        "properties": "InstanceData<AcceptConcepts...>",
        "instanceName": { "type": "string", "pattern": "^s:[a-zA-Z][a-zA-Z0-9_]*$" }
      }
    }
  }}
}
```

Listing 21: The `InstanceBase` and `Instance` *ValueDomain* definitions. `Instance` alters `InstanceBase`'s schema by replacing `InstanceDataBase` with `InstanceData<AcceptConcepts...>`, so its `AcceptConcepts` fix the expected *Type* of the defined `"properties"` keys. Instance names are letter-starting strings with the remaining characters being alphanumeric or '_'.

6.3.1 Template Parameters

Both variadic groups are constrained by `"And(Concept, Not(ValueDomain))"`: their arguments range over the descendants of the hierarchy root that are not *ValueDomains* — that is, over exactly the *DomainConcepts*. The two extremes of that range collapse onto `InstanceBase`: `Instance<>` passes no concept and `Instance<Concept>` passes only the root, which (usually) defines no properties, so `InstanceData<>` and `InstanceData<Concept>` constrain nothing and coincide with `InstanceDataBase`. The `"variadicGroupIdentifiers"` of subsubsection 5.2.3 assigns, in a type application, every unprefixed argument to `AcceptConcepts`'s variadic group and every `!`-prefixed argument to `RejectConcepts`'s (Definitions 5.7 and 5.8).

6.3.2 Instantiation Arguments

An `InstanceBase` or `Instance` instantiation admits three arguments. `"properties"` holds the property and function values, as subsection 6.2 describes; `"instanceName"` is the instance's identifier; and `"concepts"` is the list of concepts that the instance declares it must be interpreted under, defaulting to the empty list. The two lists of concepts an `Instance` type application involves are written by different parties and answer different questions. `AcceptConcepts` is written at the site consuming a value and states what that site requires of it — the affordances [Gibson(1979)] the value must offer to be used there. A `Grasp` action types its `objectToGrasp` property with `"valueDomain"` `Instance<Graspable>` and thereby demands of a value not that it announce itself a `Graspable` but that it meet `Graspable`'s requirements. `"concepts"` is written at the site producing the value and states how it is to be interpreted: which of the Concept Hierarchy's `"initialization"`, `"consolidation"`, `"hooks"`, `"computations"`, and `"constraint"` definitions bear on it (section 4). An interpretation is imposed and not recognized, the definitions it brings being behavior, and behavior not being inferable from the values it acts upon: an instantiation binding no property value but giving `"concepts"` `[Object]` is an `Object` instance whose property values are unknown, carrying `Object`'s hooks, constraints, and closed-world defaults, and not an instance of nothing. `"concepts"` is accordingly neither required to be exhaustive, nor required to agree with the `AcceptConcepts` of the *Type* at which the instantiation is written, nor required to be non-empty; it is required only to be internally coherent, its entries naming *DomainConcepts* that are pairwise non-disjoint and that do not ambiguously specialize a common keyword (subsection 4.2). Nor is the interpretation fixed at instantiation: the set of *DomainConcepts* an instance is interpreted under is state that the instance carries, which `"concepts"` initializes and which a Concept Hierarchy program may extend or revoke as the instance's property values change. What such a change does, and at

which points a value's state is checked against the *AcceptConcepts* of the *Type* it is written at, belong to a Concept Hierarchy's execution and are outside the scope of this document (subsection 1.2).

6.3.3 Instance Names

"*instanceName*" names the domain instance the Instance represents, and it names it in the same space as the Concept Hierarchy's variables rather than in a domain-level space of its own. In the Concept Hierarchy, an instance is referred to, e.g. when defining the value of a property whose *Type* is an Instance application, through an ordinary variable reference (section 8). Where both names are present, they must therefore agree: an instantiation bound under **"instances"** must give an "*instanceName*" equal to its binding name, or omit it and let the binding name supply it (section 9). Two Instance<*AcceptConcept..., RejectConcepts...*> values carrying the same "*instanceName*" represent the same domain entity even where they bind different property values — one may record what is known of an entity at one point, another what is known of it at another.

6.3.4 Using The Reification

The Instance family lets a property restrict its values by the affordances [Gibson(1979)] an instance must offer. A Grasp action, for example, can type its objectToGrasp property with **"valueDomain"** Instance<Graspable>, admitting exactly those instances that meet Graspable's requirements — a reification of Graspable in the sense of definition 6.1. The *RejectConcepts* group excises part of the *DomainConcept* hierarchy: a Zoo may describe its free-flying animals as Instance<Bird, !Penguin>, i.e. every Bird instance that is not a Penguin [Costinescu et al.(2025)]. This second application reifies no concept (there is no Concept Hierarchy *DomainConcept* having non-penguin birds for its requirements). It hints at the expressive power of the family, that it is not merely a *Type* per *DomainConcept*.

6.4 Disjunction: Variant<*T...*>

A single Instance<*AcceptConcepts..., RejectConcepts...*> imposes the requirements of all its *AcceptConcepts* at once. There is no way to write an Instance-*Type* that meets the requirements of Bird *or* those of Fish. The Variant<*T...*> *ValueDomain* of Listing 22 supplies the missing outer disjunction.

```
{
  |...|,
  "Variant": { "directParents": ["ValueDomain"], "data": {
    "templateContext": { "order": ["T..."], "variadicGroupIdentifiers": { "T": "" } },
    "instantiation": { "anyOf": ["T..."] }
  }}
}
```

Listing 22: The Variant<*T...*> *ValueDomain* definition. Unlike Instance, its variadic template parameter ranges over *ValueDomain Types* rather than *DomainConcepts*. A Variant- type application unifies any *Types* and is not specific to reification.

The single variadic parameter *T* is (implicitly) constrained to the descendants of ValueDomain (subsubsection 5.1.3), so a Variant type application bears on *DomainConcepts* only when its arguments are Instance type applications. Its **"instantiation"** admits a JSON value if that value instantiates any one of the schemas of its template arguments: the pack *T...* expands into the **"anyOf"** array by definition 5.16, so that a type application binding *T* to ["String", "Number"] monomorphizes the schema to { **"anyOf"**: ["String", "Number"] }.

6.4.1 Expressive Power Of The Family

A Concept Hierarchy can already form conjunctions of *DomainConcepts* without any of the above: a *DomainConcept* with **"directParents"**: [A, B] imposes the requirements of both, up to whatever it adds of its own. What it cannot form is a complement, complementation being absent from the axioms of section 4; and what it cannot do in either case is form the set *at the site where the Type is needed*, rather than by extending the ontology with a concept that exists only to be named there. This is what the template parameters can express, and it is why

the reification takes concepts as arguments instead of giving each *DomainConcept* a *Type* of its own: `AcceptConcepts` intersects, `RejectConcepts` complements, and `Variant<...>`'s `T` unites. Every Boolean combination of the Concept Hierarchy's *DomainConcepts* can be written as a `Variant<...>` of `Instance` type applications, and no further construct is needed to reach one. Subsection 6.5 states this precisely and proves it.

6.5 Admitted Instances, Reification, and Expressive Completeness

The preceding subsections give the syntax of the reifying family. This one states which instances a ground type application of that family admits, which applications reify a *DomainConcept* in the sense of definition 6.1, and that the family reaches every Boolean combination of the Concept Hierarchy's *DomainConcepts*. Write $[\![C]\!]$ for the set of instances satisfying the requirements of a *DomainConcept* C, and $[\![\tau]\!]$ for the set of values a ground type application τ admits.

Definition 6.2 (Admitted instances)**.** For $\sigma = \texttt{Instance<}A_1, \ldots, A_m, !R_1, \ldots, !R_n\texttt{>}$ and ground type applications $T_1, \ldots, T_k$,

$$[\![\sigma]\!] = \bigcap_{i=1}^{m} [\![A_i]\!] \setminus \bigcup_{j=1}^{n} [\![R_j]\!], \qquad [\![\texttt{Variant<}T_1, \ldots, T_k\texttt{>}]\!] = \bigcup_{j=1}^{k} [\![T_j]\!].$$

The empty intersection is the set of all *DomainConcept* instances, so $[\![\texttt{Instance<>}]\!] = [\![\texttt{InstanceBase}]\!]$ is that set; the empty union is $\emptyset$, so $[\![\texttt{Variant<>}]\!] = \emptyset$.

One property of $[\![\cdot]\!]$ is needed below and follows from the disjointness axiom rather than from definition 6.2. Declared disjointness is extensional: where two *DomainConcepts* C and D are declared distinct, $[\![C]\!] \cap [\![D]\!] = \emptyset$, no instance satisfying the requirements of one can be admissible as an instance of the other. This is checked whenever concept membership is checked.

Proposition 6.3 (Which applications reify)**.** *A ground type application τ of the family reifies a DomainConcept C if and only if $[\![\tau]\!] = [\![C]\!]$. In particular,* `Instance<`C`>` *reifies C for every DomainConcept C, and* `InstanceBase` *— equivalently* `Instance<>` *— admits every DomainConcept instance. An application naming several accepted concepts, or any rejected concept, reifies a concept only where the Concept Hierarchy defines one whose requirements coincide with the combination.*

Proof. definition 6.1 requires of τ that it be writable wherever a *Type* is expected, and that it admit exactly the values satisfying C's requirements, omitting none and adding none. The former holds of every ground type application of the family, all of which are *ValueDomain Types*, so the definition reduces to the latter, which is $[\![\tau]\!] = [\![C]\!]$. For $\tau =$ `Instance<`C`>`, definition 6.2 gives an intersection over the single set $[\![C]\!]$ and an empty union, hence $[\![\tau]\!] = [\![C]\!]$. For $\tau =$ `InstanceBase`, both are empty and $[\![\tau]\!]$ is the set of all *DomainConcept* instances, which is what definition 6.2 assigns to the empty intersection. Every remaining application denotes a Boolean combination of concept extensions by definition 6.2, and such a combination equals some $[\![C]\!]$ only where the Concept Hierarchy defines that C. □

The `Instance<Bird, !Penguin>` of subsection 6.3 is an application of the last kind: it reifies no concept, there being no *DomainConcept* whose requirements are those of a non-penguin bird, yet it denotes a well-defined set of instances. The family is therefore not one *Type* per *DomainConcept*, and the following states how much more it is.

Proposition 6.4 (Expressive completeness)**.** *Let $\mathcal{B}$ be the Boolean subalgebra generated by the extensions $[\![C]\!]$ of the Concept Hierarchy's DomainConcepts, over the set of all DomainConcept instances. Every member of $\mathcal{B}$ is $[\![\tau]\!]$ for some ground type application $\tau =$* `Variant<`$\sigma_1, \ldots, \sigma_k$`>` *whose arguments are* `Instance` *type applications, and every such τ denotes a member of $\mathcal{B}$.*

Proof. By definition 6.2, an `Instance` application denotes an intersection of concept extensions with the complements of concept extensions — a conjunctive clause over concept membership — and a `Variant` application denotes the union of its arguments' denotations. A `Variant` of `Instance` applications is therefore a disjunction of such clauses, which is a member of $\mathcal{B}$; this is the second claim. For the first, take a member of $\mathcal{B}$ and write it as a Boolean formula over the generators. Since $\{\wedge, \vee, \neg\}$ is functionally complete, the formula has a disjunctive normal form, and translating each clause's positive literals into `AcceptConcepts`, its negative literals into `RejectConcepts`, and the disjunction into the `T` group of `Variant<...>` yields a τ with the required denotation. □

The two translations of the proof are mutually inverse up to the ordering and duplication of arguments, so the correspondence between disjunctive normal forms and `Variant`-of-`Instance` applications is a syntactic bijection. The map to admitted sets is nonetheless surjective onto $\mathcal{B}$ without being injective, since subsumption and disjointness collapse distinct applications onto one set: `Instance<Bird, Penguin>` and `Instance<Penguin>` are one such pair. This is what subsection 6.4 asserts of the three template parameter groups: *`AcceptConcepts`* intersects, *`RejectConcepts`* complements, and `Variant`'s *`T`* unites. No other construct is needed to reach a member of $\mathcal{B}$.

6.6 Subtyping the Reifying *Types*

The subtype relation $\sqsubseteq_T$ of subsubsection 5.3.3 is induced by the template substitutions a *ValueDomain* declares for its parents' template parameters. That mechanism relates a *ValueDomain* to its ancestors, e.g. `Instance<Bird>` $\sqsubseteq_T$ `InstanceBase`, but it does not relate two applications of `Instance` to each other, e.g. `Instance<Penguin>` and `Instance<Bird>`, and it does not relate `Instance<Bird>` to any `Variant<...>` that includes it. Yet every `Penguin` instance is a `Bird` instance, and a value of `Instance<Penguin>` must therefore be usable wherever `Instance<Bird>` is expected. The subsumption that makes it usable — `Penguin` beneath `Bird` — is recorded in the *DomainConcept* hierarchy, which template substitution does not consult. `Instance` therefore needs its own subtype relation, one that directly encodes subsumption. Definitions 6.5 and 6.6 extend the $\sqsubseteq_T$ relation to cover the aforementioned cases[17].

Definition 6.5 (The subtype relation for `Instance` type applications)**.** Let $\sigma =$ `Instance<`$A_1, \ldots, A_m, !R_1, \ldots, !R_n$`>` and $\sigma' =$ `Instance<`$A'_1, \ldots, A'_{m'}, !R'_1, \ldots, !R'_{n'}$`>`, and write $C \sqsubseteq D$ for subsumption of *DomainConcepts* and $C \perp D$ for their disjointness. Then σ is a subtype of σ', written as $\sigma \sqsubseteq_T \sigma'$, if

1. for every $k \in \{1, \ldots, m'\}$ there is an $i \in \{1, \ldots, m\}$ with $A_i \sqsubseteq A'_k$; and
2. for every $l \in \{1, \ldots, n'\}$ there is either a $j \in \{1, \ldots, n\}$ with $R'_l \sqsubseteq R_j$, or an $i \in \{1, \ldots, m\}$ with $A_i \perp R'_l$.

Condition (1) requires each concept that σ' accepts to be entailed by one that σ already accepts, and condition (2) requires each concept that σ' rejects to be excluded by σ — either because σ rejects a concept subsuming it, or because σ accepts a concept disjoint from it. Both are decided from the *DomainConcept* hierarchy alone, so the relation is checkable statically, as subsection 8.5 requires of the Exact and Sub distinction.

Definition 6.6 (The subtype relation for `Variant` type applications)**.** Let τ be any ground type application. Then

1. $\tau \sqsubseteq_T$ `Variant<`$T_1, \ldots, T_k$`>` if $\tau \sqsubseteq_T T_j$ for some $j \in \{1, \ldots, k\}$; and
2. `Variant<`$S_1, \ldots, S_p$`>` $\sqsubseteq_T \tau$ if $S_i \sqsubseteq_T \tau$ for every $i \in \{1, \ldots, p\}$.

Condition (1) admits a value of any one disjunct where the union is expected, and condition (2) allows a union to be used where every disjunct would be. Applying them in turn relates two `Variant<...>` applications: `Variant<`$S_1, \ldots, S_p$`>` $\sqsubseteq_T$ `Variant<`$T_1, \ldots, T_k$`>` whenever every S_i is a subtype of some T_j. The empty `Variant<>` is a subtype of every τ by (2), vacuously.

Definition 6.7 (Closure of the subtype relation)**.** Beyond the reflexive-transitive closure of $\lhd$ (subsubsection 5.3.3), $\sqsubseteq_T$ is the least relation additionally closed under Definitions 6.5 and 6.6.

Closure is what makes the rules combine: `Instance<Penguin>` $\sqsubseteq_T$ `Variant<Instance<Bird>, String>` is derived by definition 6.5 from `Penguin` $\sqsubseteq$ `Bird`, then by definition 6.6, and neither rule reaches it alone. The relation is a preorder and not a partial order: `Instance<Bird, Bird>` and `Instance<Bird>` are mutual subtypes, as are any two applications differing only by repeated arguments. Both rules decide a subtype question by looking up subsumption and disjointness in the *DomainConcept* hierarchy, performing a number of lookups quadratic in the arities of the two applications, and consulting nothing else — neither the instance database nor a reasoner over the concepts they name. This is what keeps the check static and what limits it: Definitions 6.5 and 6.6 are sound with respect to the sets that the two families admit (definition 6.2), and complete with respect to neither.

[17] Definition 6.5 is not a variance rule. It relates applications of unequal arity and appeals to declared disjointness as well as to subsumption, and no annotation of `Instance`'s template parameters as covariant or contravariant would yield it.

Proposition 6.8 (Soundness of the subtype rules)**.** *If $\sigma \sqsubseteq_T \sigma'$ holds by definition 6.5 or definition 6.6, then $[\![\sigma]\!] \subseteq [\![\sigma']\!]$.*

Proof. Let σ and σ' be as in definition 6.5 and let $x \in [\![\sigma]\!]$. For every k, condition (1) supplies an A_i with $A_i \sqsubseteq A'_k$; since $x \in [\![A_i]\!]$ and subsumption is inclusion of extensions, $x \in [\![A'_k]\!]$. For every l, condition (2) supplies either an R_j with $R'_l \sqsubseteq R_j$, in which case $x \notin [\![R_j]\!] \supseteq [\![R'_l]\!]$, or an A_i with $A_i \perp R'_l$, in which case $x \in [\![A_i]\!]$ and, by the extensionality of declared disjointness above, $[\![A_i]\!] \cap [\![R'_l]\!] = \emptyset$, giving $x \notin [\![R'_l]\!]$. Hence $x \in [\![\sigma']\!]$ by definition 6.2. For definition 6.6, condition (1) gives $[\![\tau]\!] \subseteq [\![T_j]\!] \subseteq \bigcup_j [\![T_j]\!]$, and condition (2) gives $\bigcup_i [\![S_i]\!] \subseteq [\![\tau]\!]$ from $[\![S_i]\!] \subseteq [\![\tau]\!]$ for every i. The closure of definition 6.7 preserves the inclusion, and transitivity of $\subseteq$ discharges the transitive step. □

Incompleteness follows from what the two rules do not consult. Both decide by subsumption and disjointness lookups alone, so a containment holding by virtue of a covering declaration is not derived: with `Bird` declaring `"directChildren": [Penguin, Sparrow]`, every `Bird` instance is a `Penguin` or a `Sparrow` instance, so `Instance<Bird>` and `Variant<Instance<Penguin>, Instance<Sparrow>>` admit exactly the same instances, yet neither definition 6.5 nor definition 6.6 relates them in either direction. The Concept Hierarchy accepts this. A rule closing the gap would replace a check quadratic in the arities of the two applications with reasoning over the covering and disjointness declarations of the hierarchy as a whole, and the resulting check would no longer be decidable from the two applications and the *DomainConcept* hierarchy's direct relations alone.

7 Definition of *Functions*

A *Function* is the only *ValueDomain* that is also callable, and the only construct able to change the value of a *ValueDomain* instance. Its classification as a *ValueDomain* reflects direct instantiability: every non-abstract *ValueDomain* may be instantiated directly[18]. Because *Functions* are stateless, i.e. they retain no data between evaluations, they require no instantiation parameters, and therefore keep the default *ValueDomain* instantiation schema `{"type": "object", "maxProperties": 0}`.

Three operations are defined on a *Function*: it may be instantiated, evaluated, or composed. Evaluation and composition are the common cases; instantiation is rare. Beyond the keywords a *ValueDomain*'s **`data`** may carry (section 5), a *Function*'s **`data`** admits four additional keywords: **`"interface"`**, **`"procedure"`**, **`"addNewVariablesInExistingScope"`**, and **`"subScopes"`** (Listing 23).

```
{
  |...|,
  "FunctionExample": {  "data": {
    "templateContext": |as in ValueDomain definition data|,
    "interface": {
      "arg1": |string: type application|,
      "arg2": [|string: type application|, |access character|, |provenance character|],
      |...|,
      "res": [|string: type application|, |res access character|, |res provenance character|],
      "_defaultArgumentValues": {
        |string: argument name from "interface"|: |argument Type expr|,
        |...|
      }
    },
    "procedure": |FunctionComposition expr|,
    "addNewVariablesInExistingScope": {
      |string: variable name|: |string: type application|,
      |string: variable name|: [|string: type application|, false],
      |string: Function argument name|: [|string: type application|, true],
      |...|
    },
    "subScopes": {
```

[18] A *DomainConcept* is instantiated only via reification through the `InstanceBase` *ValueDomain*—or its template subconcept `Instance<C>`, which admits only instances of `C`, see section 6

```
      |string: argument name from "interface" ≡ scope arg name|: {
        |string: variable name|: |string: type application|,
        |string: variable name|: [|string: type application|, false],
        |string: Function argument name (≠ scope arg name)|: [|string: type application|, true],
        |...|
      }
    }
  }}
}
```

Listing 23: The "template" definition of *Function* `data`. `res` is defined in a *Function* only when it has a result.

7.1 The *Function* `interface`

A *Function*'s evaluation contract is declared in the `"interface"` keyword of its `data`, which fixes the *Function*'s arguments and its result type.

Definition 7.1 (Function interface)**.** The `"interface"` is a `JSON object` whose keys are the *Function*'s argument names—each a lowercase-initial identifier—mapped to *argument type definitions*, together with the reserved key `"res"` giving the result type. Argument order is immaterial, both in the interface and at evaluation. An optional `"_defaultArgumentValues"` `JSON object`—reserved by its underscore prefix, which no argument name may bear—maps a subset of the arguments to default value expressions; an argument that has a default may be omitted at evaluation. Each default expression must satisfy the declared *Type* and characters of its argument and may reference other arguments of the same *Function*.

Definition 7.2 (Argument type definition)**.** An argument is bound to its *Type* in one of two forms. The *shorthand* is a single string naming the expected *Type*. The *canonical* form is an array of one to three elements listing, in order, the expected *Type*, the expected *access* character, and the expected *provenance* character; omitted trailing elements take their defaults[19]. The result key `"res"` is bound analogously, but over a restricted character set. Table 9 states the admissible characters and defaults for each position.

One *Type* is inadmissible at the `"res"` position: `FunctionComposition`, and every subtype of it, since a composition may not escape the construct that holds it (subsubsection 7.3.3). The restriction is on `"res"` alone; a *Function argument* may be composition-typed, and `FunctionCompositionRes<T>` is exactly that case (subsubsection 7.3.1).

Definition 7.3 (Interface inheritance)**.** The `"interface"` is inherited by subconcepts, which may add new arguments or override the `"_defaultArgumentValues"` of inherited ones, but may not remove an argument. To keep inheritance unambiguous, a *Function* may have at most one direct parent: two parents could otherwise declare a same-named argument with differing *Type*, access, or provenance characters, and the Concept Hierarchy does not yet define a rule for merging conflicting argument definitions.

Inherited `"_defaultArgumentValues"` are combined per argument and not as a whole: a *Function* giving a default for an argument supersedes the default its parent gives for that same argument, while the defaults of arguments it does not mention are inherited unchanged. Because a *Function* has at most one direct parent, the chain of contributing definitions is linear, and this rule fixes exactly one default expression per argument, leaving no order of combination to settle. These inter-argument references form a dependency graph among the defaults which need not be acyclic at the *Function* definition: a reference-cycle between two default expressions is broken by any *Function* call that supplies one of the two arguments, and is ill-formed only for a particular evaluation or composition that supplies neither. Whether the supplied arguments admit a resolution order is therefore a condition on an individual evaluation and not on the *Function* definition.

Definition 7.4 (Argument-supply well-formedness)**.** Let F be a *Function*, let S be the set of arguments supplied by an evaluation or composition of F, and let U be the remaining arguments of F's `"interface"`. The evaluation is well-formed with respect to argument supply if and only if

[19] The element order here differs from that of the two-element array shorthand of an `"instantiation"` schema leaf (subsubsection 5.4.1), whose second element is the *provenance* character: an `"instantiation"` leaf declares no access character, an `"interface"` position declares both, and the access character comes first.

Table 9: Admissible and default access and provenance characters, by interface position. **"res"** excludes **GetModify** and ***ResetAddr***, since a result is either read or written but never read-and-written, and never merely reset.

Position	Access characters	Provenance characters	Default (access, provenance)
argument	**Get**, **Modify**, **GetModify**	***Any***, ***Addr***, ***ResetAddr***	**Get**, ***Any***
"res"	**Get**, **Modify**	***Any***, ***Addr***	**Modify**, ***Any***

1. every argument in U has an entry in **"_defaultArgumentValues"**; and
2. the relation on U relating u to u' whenever u's default expression references u' is acyclic.

Condition (2) is equivalent to the existence of a topological order of U in which every default expression references only arguments of S and arguments preceding it, which is the order in which the defaults are resolved. A reference-cycle among the defaults of F therefore invalidates only those evaluations supplying no argument of the cycle.

ResetAddr strengthens ***Addr***: the supplied expression must be addressable in exactly the same sense, and the *Function* additionally discards whatever the addressable value held before its procedure begins. The distinction constrains what the *Function* does with the address, not what may be written at the site. Thus, it is admissible at an argument position only, a result being produced rather than reset.

Definition 7.5 (Evaluability)**.** A *Function* is *evaluable* iff it is non-abstract and its (possibly inherited) **"interface"** defines **"res"**. An abstract *Function* may declare an **"interface"** for its subconcepts to inherit, but can be neither instantiated nor evaluated.

7.2 *Function* Evaluation

The syntax of a *Function*-evaluation is a single-key `JSON object` whose key is the *Function*'s type application and whose value maps argument names to argument-value expressions, see Listing 24. Each such expression is checked against the declared *Type*, access, and provenance characters of its argument (section 8); arguments with a default value specified in **"_defaultArgumentValues"** may be omitted from the evaluation. The evaluation contract is that the *Function* returns a **"res"** *Type* value; the contract is not semantic: no guarantees on the value other than its *Type* are given.

```
{
  |string: Function type application|:  {
    |string: Function argument name|:  |argument Type expr|,
    |...|
  }
}
```

Listing 24: The syntax for a *Function* evaluation. Expressions are colored in `dark red`. *Function* evaluations are expressions themselves and can contain as sub-expressions other *Function* evaluations.

The variables that a *Function* evaluation can use are global variables and any local variables that are available in the scope at the point of writing the *Function* evaluation. When *Function* evaluations are written as argument values to another *Function* F, they are evaluated at the point where F is evaluated, not when the argument of F is deserialized.

7.3 *Function* Composition

A *Function* composition is a value denoting a procedure: a *Function* together with the expressions supplying its arguments, written down but not thereby run. It is written with exactly the evaluation syntax of subsection 7.2, subject to one relaxation: a *Function* that returns no value may be composed, whereas an evaluation must yield a **"res"** value (definition 7.5).

A composition is not itself evaluable (subsubsection 7.3.5); it is run by whichever construct holds it, and how often is that construct's affair rather than the composition's. A **hooks** body is run when the position it is attached

to is reached; a `computations` derivation is run when the property is re-derived; a composition supplied at a deferred *Function* argument, below, is run by that *Function*'s procedure. This is what separates a composition from an evaluation: an evaluation is performed once and produces one value, whereas a composition produces a value on each run and need not be run at all.

7.3.1 Evaluating Argument Expressions

The expressions supplying a *Function*'s arguments are evaluated when the *Function* evaluation to which they belong is itself performed, and a *Function* evaluation nested inside such an expression is also performed at that moment.

By default, an argument expression is evaluated exactly once per evaluation of the *Function* it is supplied to, before that *Function*'s procedure begins. The procedure receives a value, and not a means of obtaining one: consulting the argument several times consults that one value. A procedure requiring the opposite — consulting an argument zero, one, or several times, and obtaining a value produced afresh on each consultation — must say so in its `"interface"`, by declaring that argument with the *Type* `FunctionCompositionRes<T>`, where `T` is the *Type* of the value each consultation produces (Listing 25). We call an argument so declared *deferred*, and every other argument *eager*. The argument declaration is what makes the deferral visible to the caller: whether an argument's value is fixed once per call or produced afresh per consultation is read off the *Function*'s interface, and never off the expression supplied at the call site.

The declaration does not, however, decide that each consultation recomputes; it only permits it, and the caller decides. `FunctionCompositionRes<T>` instantiates as either of two forms, per its `"instantiation"` schema `{"anyOf": ["T", "FunctionComposition"]}`:

- the *value form*, an expression of *Type* `T`. It is an ordinary eager argument expression: it is evaluated once, when the enclosing *Function*'s arguments are bound on entry to its procedure, and the resulting value is held for the whole of that call, so every consultation yields it unchanged.
- the *composition form*, a `FunctionComposition`. Nothing is evaluated when the argument is bound; the composition is run once per consultation, and successive consultations may yield different values.

The distinction is therefore between an argument that *may* be recomputed and one that *is*, and `FunctionCompositionRes<T>` is the *ValueDomain* that unifies the two, so that a procedure written against it is indifferent to which the caller supplied.

Running a composition performs its procedure anew. *Function* evaluations written inside the composition are performed anew on every run as well: nothing is cached between successive runs of a *Function* composition. Between two runs, the variables that the composition names may have changed, which is why successive runs of a composition can produce different values. The value form's single evaluation is not an exception: it is an evaluation of the enclosing *Function*'s argument and not a run of any composition.

7.3.2 Differentiating Between *Function* Evaluation And Composition: `isFunctionEvaluation`

The two forms of `FunctionCompositionRes<T>` are not always distinguishable from the `JSON` alone. A `JSON object` keyed by a *Function* type application matches both branches of the `"anyOf"`. Read as a *Function* evaluation, it is an expression of *Type* `T`, provided the *Function*'s `"res"` *Type* is a subtype of `T`. Read as a composition literal, it is a `FunctionComposition`. The `"isFunctionEvaluation"` keyword, written beside the *Function*'s name in the then two-key `JSON object`, selects between them. An absent or `false` value selects the composition form, which is therefore the default; `true` selects the value form, the object then being classified as a *Function* evaluation at expected *Type* `T` and subject to the ordinary admissibility rules of subsection 8.5.

The `"anyOf"` of `FunctionCompositionRes<T>` is deliberately not a `"oneOf"`: neither branch mentions the flag, so an object composing a *Function* whose `"res"` is a subtype of `T` matches both, flagged or not. Which branch it is read under is fixed by the rule above and not by the schema.

Supplying the composition form makes each consultation reflect the state of the variables the composition names at the moment of that consultation; supplying the value form fixes the value once, on entry to the procedure. A loop condition written in the composition form terminates the loop when the properties it reads change; the same expression written in the value form does not, being evaluated once before the first iteration.

The keyword is required only where a genuine ambiguity arises, and there are exactly two such positions. This is one; the other is an argument-less *Function* at a site expecting a *Function* instantiation value (subsection 7.4). At a site whose expected *Type* is `FunctionComposition` itself — a `hooks` body, a `computations` derivation, a *Function*'s `"procedure"` — no ambiguity arises, there being no `T` for an evaluation to produce. Such an object is always a composition literal, and it is a well-formedness violation to write `"isFunctionEvaluation": true` there (no *Function* evaluation returns a `FunctionComposition` value). A `false` flag is admitted and carries no information.

7.3.3 Compositions Are Not Closures

A composition is an open term over names. It records the *Function* to run and the expressions supplying its arguments, and those expressions may name variables; what it does not record is any binding for them. A composition captures no environment: the names it contains are resolved when it is run, against the scope then in force, and not against the scope in which it was written.

Two restrictions follow, and both restrict where a `FunctionComposition` value may go rather than what it may contain. A `FunctionComposition` may not be bound to a variable, and may not be a *Function*'s `"res"` *Type* — so that neither a (local or global) variable of `FunctionComposition` *Type* nor a `Return<FunctionComposition>` is writable. Either would let the value outlive the scope its names were written against and be run where those names are bound differently, or not at all. A composition therefore travels no further than the construct that holds it: it is written at the position at which it is run, i.e. as the value of the `Evaluate` and `EvaluateRes<T>` family of *Functions*, or supplied directly at the deferred argument of the *Function* that runs it. A procedure that is to be stored, passed, and run elsewhere is written as a `CustomFunction` instead (subsubsection 7.3.5), whose `"interface"` names every value its `"procedure"` depends on and thereby closes the term, capturing nothing.

7.3.4 `FunctionComposition` Instantiation Schema

Unlike an ordinary *ValueDomain*, the keys a `FunctionComposition` value admits are not fixed: they are the arguments of whichever *Function* is being composed, and so vary from one composition to the next. Draft-07 cannot state such a value-dependent key set, so `FunctionComposition` relies on a final extension to the schema dialect—the `"args"` form of the `"properties"` keyword (Listing 25). The schema nests two `JSON object` schemas. The outer schema is a single-key wrapper (`"minProperties"` and `"maxProperties"` both 1) whose sole key is constrained by `"propertyNames"`—through `"format": "Type"` and `"constraint": "Function"` (subsection 5.4)—to a type application of a subtype of `Function`[20]. The inner schema—the value of that sole key—sets `"properties"` to the string `"args"`, which resolves against the *Function* type application named by the enclosing key (a read one nesting level up) and expands to exactly that *Function*'s arguments, each stamped with the provenance and access characters its `"interface"` declares. Pairing `"args"` with `"additionalProperties": false` makes the expansion exhaustive—a composition may supply the composed *Function*'s arguments and no other key. The reserved `"res"` key is not among them, since `"args"` matches arguments alone and the result type remains bound in the `"interface"`.Which arguments a composition may omit is, likewise, not fixed by this schema but by the composed *Function*'s `"_defaultArgumentValues"`: an argument with a default is optional, one without is required.

```
"FunctionComposition": { "directParents": ["ValueDomain"], "data": { "instantiation": {
  "type": "object",
  "minProperties": 1, "maxProperties": 1,
  "propertyNames": { "type": "string", "format": "Type", "constraint": "Function" },
  "additionalProperties": {"type": "object", "properties": "args", "additionalProperties": false}
}}},
"FunctionCompositionRes": { "directParents": ["FunctionComposition"], "data": {
  "templateContext": ["T"], "instantiation": {"anyOf": ["T", "FunctionComposition"]}
}}
```

Listing 25: The `FunctionComposition` *ValueDomain* definition, including its `instantiation` schema, and its subconcept `FunctionCompositionRes`, which is expected to produce a value of *Type* `T`.

[20]The `"Type"` format, rather than `"Concept"`, is required so that a parameterized *Function* such as `Assign<Boolean>` is also admitted.

To note is that FunctionCompositionRes<*T*> is a subtype of FunctionComposition, but its schema accepts a larger set of JSON values than that of FunctionComposition (subsubsection 5.4.10). FunctionCompositionRes<*T*> denotes the *Functions* that return a value of *Type T* on every consultation, a subset of the *Function* compositions that FunctionComposition denotes. FunctionCompositionRes's schema admits one JSON form that its supertype does not — an expression of *Type T*, which denotes a constant *Function* composition returning that value.

7.3.5 CustomFunction and FunctionComposition

The FunctionComposition *ValueDomain* represents a composition of *Functions* but is not itself evaluable. It becomes evaluable through the CustomFunction *ValueDomain*, whose instantiation pairs an "*interface*" with a "*procedure*" of *Type* FunctionComposition (Listing 26). A bare FunctionComposition used as a CustomFunction value denotes a custom *Function* with an empty interface, i.e. no arguments. A "*procedure*" may reference only global variables and the argument names of its "*interface*". Whether a CustomFunction returns a value is not declared in its *interface*. This is because the evaluation of a CustomFunction, and thus its result value, is controlled by the EvaluateFunction and EvaluateFunctionRes<*T*> family of *Functions*. Declaring the **"res"** keyword in a CustomFunction's *interface* would be redundant. The CustomFunction argument values may contain non-ground type applications when instantiated within a template context that supplies the free parameters.

```
{
  "oneOf": [ "FunctionComposition", {
    "type": "object", "additionalProperties": false,
    "properties": {
      "interface": {
        "type": "object", "additionalProperties": false,
        "properties": {
          "_defaultArgumentValues": { "type": "object",
                                      "patternProperties": { "^[a-z][A-Za-z0-9_]*$": true } }
        },
        "patternProperties": { "^[a-z][A-Za-z0-9_]*$": { "$ref": "#/$defs/argumentTypeDefinition"} }
      },
      "procedure": "FunctionComposition"
    },
    "required": ["procedure"]
  }],
  "$defs": {
    "argumentTypeDefinition": {
      "oneOf": [ "TypeValue", {
        "type": "array", "minItems": 1, "maxItems": 3,
        "items": [ "TypeValue", { "$ref": "#/$defs/argAcc" }, { "$ref": "#/$defs/argProv" } ]
      }]
    },
    "argProv": { "type": "string", "enum": ["Any", "Addr", "ResetAddr"] },
    "argAcc":  { "type": "string", "enum": ["Get", "Modify", "GetModify"] }
  }
}
```

Listing 26: The instantiation JSON Schema for the CustomFunction *ValueDomain*. TypeValue was defined in subsection 6.1.

7.4 *Function* Instantiation

Instantiating a *Function* yields a variable of *Function Type*. This is the rarest of the three operations: a *Function* variable carries no arguments and cannot be invoked through the evaluation syntax above. Invoking it requires an *application-supplied* higher-order *Function*—an EvaluateFunctionRes<*T*> that takes an instantiated *Function* together with the argument values named in its **"interface"**, applies them, and returns the *Function*'s *T*-typed result. Such a *Function* is not built into the Concept Hierarchy; the hierarchy author must define it. Its **interface** could be {"*function*": "Function", "*arguments*": "Map<String, ValueDomain>"}. An analogous author-

supplied *Function* evaluates a `CustomFunction` value (subsubsection 7.3.5) by running its `"procedure"` against the values of its arguments named in its `"interface"`.

7.5 The *Function* `procedure`

The **`"procedure"`** keyword of a *Function*'s **`data`** is the concept-level counterpart of a `CustomFunction` value: it defines the *Function*'s implementation as a `FunctionComposition` of already-declared *Functions*, obeying the same scoping rule. Recursion is expressed via evaluating the same *Function* inside its **`"procedure"`**. As with a `CustomFunction` `"procedure"`, a **`"procedure"`** may contain non-ground type applications when its *Function* is parameterized.

7.6 Local Variables: `addNewVariablesInExistingScope` And `subScopes`

Two **`data`** keywords let a *Function* introduce local variables: **`"addNewVariablesInExistingScope"`** adds them to the scope in which the *Function* is evaluated, whereas **`"subScopes"`** adds them to the sub-scope of a named argument, so that the expression supplying that argument may reference them. A variable introduction has a shorthand and a canonical form, where `name` is an identifier and `type` a string type application:

- shorthand: `name: "type"`;
- canonical (static naming): `name: ["type", false]`;
- canonical (dynamic naming): `name: ["type", true]`.

In the canonical form, the boolean selects between static and dynamic naming. With `false`, `name` is used literally as the new variable's name. With `true`, `name` must instead be a `String`-typed argument of the introducing *Function*, whose value—evaluated at run time—becomes the variable's name[21]. The shorthand maps onto the `false` canonical form. In either form, the declared type application may not be `FunctionComposition` or a subtype of it, for the reason given in subsubsection 7.3.3.

8 Expressions in a Concept Hierarchy

Every value that a Concept Hierarchy program supplies is written as an *expression*: the default of a property, the procedure of a hook, the argument of a *Function* evaluation, and the value stored in a global variable are all specified as expressions. Expressions are, therefore, the leaves at which the declarative scaffolding of concepts, types, and functions bottoms out in concrete data. This subsection fixes four things in turn: the small grammar of *expression forms*; the *scopes* against which the variable form is resolved; the deterministic *classification* of a raw `JSON` value into one of these forms; and the static-semantic *admissibility* rules that decide whether a classified expression is well-formed at the site where it occurs. We write $\sqsubseteq_T$ for the subtype relation on ground *ValueDomain* applications (section 5).

8.1 Expression Forms

An expression occurs at a site that expects a value of some ground *ValueDomain* type application, which we call its *expected Type*. Every expression takes one of three forms.

Definition 8.1 (Expression)**.** Let τ be a ground type application. An *expression of expected Type* τ is a `JSON` value that is one of:

1. a *ValueDomain instantiation*, denoting a freshly constructed value of a *ValueDomain*;
2. a *Function evaluation*, denoting the value produced by evaluating a *Function*; or
3. a *variable*, denoting the value currently held by a named variable in scope.

[21] Dynamic naming lets, for instance, the *Function* `IterateList<T>` name its iteration variable after the value of one of its own arguments.

Two of these forms are further distinguished for well-formedness. A *ValueDomain* instantiation is either *direct* — the `JSON` value matches the `"instantiation"` schema of τ itself — or a *narrowing*: a `JSON object` `{D: w}` whose single *content key* is a *Type* $D \sqsubseteq_T \tau$ and whose value w matches D's `"instantiation"`. A narrowing selects a subtype's instantiation where the expected *Type* is broader; it is the ordinary way to define a global variable, whose expected *Type* is always *ValueDomain*. A *Function* evaluation is either FEvalNoAddr or FEvalAddr according to the provenance character of the *Function*'s `"res"` (section 7); the two are syntactically identical, so the distinction is read off the *Function* concept, not the call site. Both the narrowing and the *Function* evaluation forms are `JSON objects` bearing a single *content key* — the *ValueDomain* or *Function* name whose value the object carries. Either may have an additional `"isFunctionEvaluation"` key, which is not a content key. It records how the object is to be read, is consulted only during classification (algorithm 1), and is removed before the object is matched against any `"instantiation"` schema. Throughout, "content key" means a key other than this `"isFunctionEvaluation"` key, so a narrowing or an evaluation `JSON object` has exactly one content key. We abbreviate the five resulting shapes as Inst, Narrow, FEvalNoAddr, FEvalAddr, and Var.

The syntax for specifying a variable is a `JSON string` identifying the variable; the syntax for a *Function* evaluation was presented in subsection 7.2 and, for *ValueDomain* instantiation, the `"instantiation"` schema of the target *ValueDomain* (section 5) defines the syntax of the `JSON` value.

8.2 Scopes and Variable Resolution

A variable expression is a bare `JSON string`, resolved against three nested namespaces. Correctly classifying a `JSON string` therefore requires knowing which names are bound at the site of the expression.

Definition 8.2 (Variable scopes)**.** The variables in scope at an expression site are the union of:

Global variables — the bindings declared under a Concept Hierarchy's `"instances"` key. They are *in scope* in every expression.

Local variables — variables introduced by a *Function* during a procedure, either in the *calling* scope via `"addNewVariablesInExistingScope"` or in an argument's *sub-scope* via `"subScopes"`. A local variable is *in scope* only within the scope in which it was declared. A variable introduced via `"addNewVariablesInExistingScope"` is furthermore *in scope* only *after* the point at which the introducing *Function* is evaluated, since it does not exist before that *Function* executes.

Implicit variables — variables bound by the enclosing construct without an explicit declaration. These are: the interface arguments of a *Function*, within that *Function*'s `"procedure"`; the arguments of a `CustomFunction`, within its `"procedure"`; the properties and functions of a *DomainConcept*, within its `"initialization"`, `"consolidation"`, `"constraint"`, `"hooks"`, and `"computations"` procedures; and, within a hook attached to a (*Function* F, argument) pair, the remaining arguments of F; a complete enumeration is given in Table 12.

Variables can be shadowed by names in deeper nested scopes. Every *Function* argument adds a new nested scope into the existing scope of the *Function* evaluation or composition and that scope is completely removed once the evaluation of the argument is finished. The scope can be thought of as a stack, with variable names being resolved from the top-most stack level to the bottom-most level, where the global variables live. Implicit variables are added to the scope on the first stack level above the global variables and thus may be shadowed by later-occurring variables and only within the scope in which the shadowing variables occur.

For a *DomainConcept* function, access to the instance's own properties and functions is mediated by a distinguished `instance` argument of the appropriate `Instance<AcceptConcepts..., RejectConcepts...>` *Type*[22]. A non-static function receives this argument by default; a `"static"` function does not have this keyword because a static definition is shared across all instances and has no implicit instance of its own.

A variable string may also be an *instance-property chain*. This is a `.`-separated `JSON string` such as `instance.member` or `instance.subInstance.member`, and so on to any depth. Its first element must be a variable in scope, of type `Instance<AcceptConcepts..., RejectConcepts...>`. Every non-terminal segment must name a property whose

[22] Equivalent to the `this` or `self` keywords in `C++` or `Python` respectively.

Algorithm 1 CLASSIFY(v, τ, Σ): recovers an expression's form from a `JSON` value v, expected *Type* τ, and scope Σ.

Require: `JSON` value v; expected *Type* τ (a ground type application); scope Σ
Ensure: the form of v, or ILL-FORMED

if v is a `JSON object` with exactly one content key k **then**
  eval $\leftarrow (\tau \not\sqsubseteq_T$ `FunctionComposition`$) \wedge$
    (v carries `"isFunctionEvaluation"`: `true` $\vee$ v has no `"isFunctionEvaluation"` entry)
  if k is a *Function* type application **and** *eval* **then**
    if res$(k) \sqsubseteq_T \tau$ **then**
      return FEVAL ▷ FEVALADDR/FEVALNOADDR per k's `"res"`
    else
      return ILL-FORMED ▷ committed to evaluation: no fall-through
    end if
  else if k is a *ValueDomain* type application $\sqsubseteq_T \tau$ whose `"instantiation"` matches the content at k **then**
    return NARROW
  end if
else if v is a `JSON string` that names a variable of Σ, or is a well-formed instance-property chain over Σ **then**
  if $\text{typ}_\Sigma(v) \sqsubseteq_T \tau$ **then**
    return VAR
  else
    return ILL-FORMED ▷ committed to the variable reading: no fall-through
  end if
end if
▷ *fall-through: an object that is neither a Function evaluation nor a ValueDomain narrowing, a string that is neither a variable nor a chain, or another JSON value*
if τ is non-abstract **and** v matches τ's `"instantiation"` **then**
  return INST ▷ direct instantiation
else if some *ValueDomain* $D \sqsubseteq_T \tau$ has a `"defaultSerialization"` of the JSON primitive type of v **then**
  return INST ▷ via D's default serialization; D is unparameterized
end if
return ILL-FORMED

own type is again an `Instance` type application, so that the .-operator can be chained further. The terminal segment names any property or function declared or inherited by one of the concepts defined in the *`AcceptConcepts`* variadic group of the immediately preceding `Instance` *Type*. A chain resolves to the VAR form.

Every variable v in scope Σ carries a *Type*, written $\text{typ}_\Sigma(v)$, against which expression classification checks the expected *Type* τ. For a global variable, this is *not* `ValueDomain`[23], but the ground type application that the variable's expression actually produces. For a local variable, it is the *Type* with which the introducing *Function* declares it, and for an implicit variable, the *Type* declared by the binding construct, e.g. an interface argument's declared *Type*, or a property's `"valueDomain"`. For an instance-property chain, it is the *Type* of the terminal segment alone: because property and function names are unique across a Concept Hierarchy (definition 4.2), that name by itself determines the *Type*, and the head and non-terminal segments serve only to establish that the chain is well-formed.

8.3 Classifying a *`JSON`* Value

The three forms are not tagged; the form of an expression is recovered from the shape of its `JSON` value together with the ambient scope, by the deterministic procedure of algorithm 1. The order of the tests matters, because the shapes overlap: a single-content-key `JSON object` could be a *Function* evaluation, a *ValueDomain* NARROW, or — by fall-through — a direct instantiation, and a `JSON string` could be either a variable name or a `String` instantiation, for example. The procedure resolves each overlap in favor of the earlier-listed form.

[23] The expected *Type* of the expression that defines the variable's value is `ValueDomain`, but this is not $\text{typ}_\Sigma(v)$ of a variable v.

Five points of the procedure warrant comment. First, at a site whose expected *Type* is not `FunctionComposition`, a single-content-key `JSON object` whose content key names a *Function* is read as an evaluation rather than a narrowing by default. The `"isFunctionEvaluation"` flag overrides this default in either direction[24]: `true` forces the evaluation reading, and `false` forces the narrowing reading. The `false` case is essential for instantiating an argument-less *Function*, whose evaluation and narrowing-instantiation syntaxes are otherwise identical: without the flag, the procedure could never reach the narrowing. Second, once v has been committed to the evaluation reading, the result type $\mathrm{res}(F)$ must be a subtype of the expected *Type* τ; if it is not, v is ill-formed, and the procedure does *not* fall through to any other reading. Third, the variable reading commits in the same way: if v is a `JSON string` naming a variable of Σ, or forming a well-formed instance-property chain, then $\mathrm{typ}_\Sigma(v) \sqsubseteq_T \tau$ decides between VAR and ILL-FORMED, and a *Type* mismatch does *not* fall through to the *ValueDomain* instantiation tests. Fourth, the direct-instantiation and default-serialization tests are the final fall-through for values that no earlier test claimed. The default-serialization test applies only to an unparameterized *ValueDomain*, since template arguments cannot in general be inferred from a bare literal. And fifth, when a parameterized *ValueDomain* defines several template-constraint-specific `"instantiation"` `JSON` schemas, the schema used to verify v is the one guarded by the first template-constraint formula that matches the governing type application — that of the expected *Type*, or of the *ValueDomain* named by a NARROW. That schema alone verifies v: if verification against it fails, matching does *not* fall through to the next template-constraint formula.

A `FunctionComposition` is a *ValueDomain*, and the `"instantiation"` dialect extended with the `"args"` form of subsubsection 7.3.4 can express that an `JSON object`'s keys are drawn from the interface arguments of the composed *Functions*, so a composition literal validates directly against that schema. Thus, a `JSON object` value at a `FunctionComposition`-typed site is always an INST expression. At a `FunctionCompositionRes<T>`-typed site, both branches of the `"anyOf"` match such an object, and the `"isFunctionEvaluation"` flag selects between them (subsubsection 7.3.2). The `JSON object` value, if it represents a *Function*, will be classified as a *Function* evaluation if `"isFunctionEvaluation": true` and a *Function* composition if the flag is missing or is `false`.

Table 10 recapitulates the cascade of algorithm 1 as a single lookup: each row pairs an expression form with the condition that selects it, in the order the conditions are tested. Reading the table top to bottom mirrors executing the algorithm, and two features of the design are visible directly in that ordering.

FEVAL and NARROW overlap because a *Function* is a *ValueDomain*: a single-key `JSON object` whose single content key names a *Function* could in principle be read either way. The algorithm breaks the tie with a default that depends on the expected *Type* and is overridable by `"isFunctionEvaluation"`. At a site whose *Type* is not `FunctionComposition`, the default is evaluation, so `"isFunctionEvaluation": false` is what one writes to instantiate (narrow) an argument-less *Function*, whose evaluation and instantiation syntaxes otherwise coincide. The same inversion applies at a `FunctionCompositionRes<T>`-typed site, where `"isFunctionEvaluation": true` is what one writes to have the *Function* evaluated once, on entry to the enclosing procedure, rather than run per consultation. At a site expecting `FunctionComposition` itself, the flag has no legitimate use, no *Function* being able to return a `FunctionComposition`. The flag is therefore needed only at these two points of genuine ambiguity.

8.4 Disambiguating `String` Literals from Variables

Because a `JSON string` is tested as a variable before it is tested as a `String` instantiation, and because the variable reading does not fall through, a `JSON string` value coinciding with an in-scope variable name is never read as a `String` literal: it is read as that variable if the variable is `String`-typed, and rejected as ILL-FORMED otherwise. To keep `String` instantiations unambiguous without resorting to the heavier NARROW form, we recommend that every `String` value be marked with an `s:` prefix, by giving `String` the instantiation schema `{ "type": "string", "pattern": "^s:" }`. Under this convention, no `JSON string` can be both a valid variable name and a valid `String` literal, and the classification of algorithm 1 is unambiguous on strings.

8.5 Static-Semantic Admissibility

Classification fixes the form of an expression; admissibility additionally depends on the provenance and access characters expected at the site, and on whether the expression's *Type* equals the expected *Type* or is a strict

[24]The `"isFunctionEvaluation"` keyword is not part of the value's content and is, therefore, removed before the value is matched against any `"instantiation"` schema, and no schema of this document, nor any a modeler writes, need mention it.

Table 10: The expression forms and the condition under which algorithm 1 assigns each, given a `JSON` value v, an expected *Type* τ (a ground *ValueDomain* application), and a scope Σ. Rows are tested top to bottom; the first whose condition holds fixes the form. A later row is reached only when every earlier condition failed, except if v is committed to the evaluation or to the variable form, where a *Type* mismatch ($\mathrm{res}(F) \not\sqsubseteq_T \tau$, resp. $\mathrm{typ}_\Sigma(v) \not\sqsubseteq_T \tau$) yields Ill-formed rather than trying a later row. Classification fixes only the *form*; admissibility with respect to the expected provenance and access characters is a separate check (Table 11).

Form	Shape of v	Condition
FEval	single-content-key `JSON object`	the key names a *Function* F that is evaluated here, *and* $\mathrm{res}(F) \sqsubseteq_T \tau$; otherwise Ill-formed
Narrow	single-content-key `JSON object` $\{D : w\}$	v is not read as an evaluation, $D \sqsubseteq_T \tau$, and w matches D's `"instantiation"`; selects the subtype D's instantiation where τ is broader
Var	`JSON string`	v names a variable of Σ, or is a well-formed `Instance...`-headed instance-property chain over Σ, *and* $\mathrm{typ}_\Sigma(v) \sqsubseteq_T \tau$; otherwise Ill-formed
Inst	any `JSON` value	τ is non-abstract and v matches τ's `"instantiation"`; or some unparameterized $D \sqsubseteq_T \tau$ has a `"defaultSerialization"` of v's `JSON` primitive type
Ill-formed	—	no condition above holds

Table 11: Static-semantic admissibility of the expression forms against the expected provenance and access characters. A ✓ marks an admissible combination and × an inadmissible one; Exact versus Sub indicates whether the expression's *Type* equals, or is a strict subtype of, the expected *Type*. Inst admits Exact because it creates a new value of the expected *Type* from an `instantiation` schema and DS, standing for a value classified into Inst via `defaultSerialization`. ***ResetAddr*** is treated the same as ***Addr*** and **GetModify** as **Modify**.

	Inst		Narrow		FEvalNoAddr		FEvalAddr		Var	
Expected character	Exact	DS	Exact	Sub	Exact	Sub	Exact	Sub	Exact	Sub
***Any*–Get**	✓	✓	✓	✓	✓	✓	✓	✓	✓	
***Any*–Modify**	✓	✓	✓	✓	✓	✓	×	✓	×	
***Addr*–Get**	×	×	×	×	×	✓	✓	✓	✓	
***Addr*–Modify**	×	×	×	×	×	✓	×	✓	×	

subtype of it. Recall (section 7) that the provenance characters are ***Any***, ***Addr*** or ***ResetAddr***, and the access characters are **Get**, **Modify** or **GetModify**. Table 11 records, for each expected (provenance, access) pair, which forms are admissible at each of the two subtyping relationships.

Two regularities in Table 11 explain the whole table. A site expecting a ***Addr*** can be satisfied only by FEvalAddr or Var, because Inst, Narrow, and FEvalNoAddr each produce a fresh, non-addressable value. And a site expecting a **Modify** rejects the strict-subtype cases of the two addressable forms: writing through an address whose *Type* strictly refines the expected *Type* could store a value inadmissible at the address's actual *Type*, so write positions are invariant in the expression *Type* rather than covariant.

8.6 Where Expressions Occur

The rules above are parameterized by the site at which an expression appears; each site fixes an expected *Type*, a provenance character, an access character, and the implicit variables in scope. Table 12 collects the sites and their parameters. A *Function*-argument site is the general case: it inherits its expected *Type* and both characters from the enclosing *Function*'s interface; the remaining sites are the specific value, predicate, and procedure positions of a concept definition.

Table 12: Expression sites, with expected *Type*, expected provenance and access characters, and available implicit variables. Specialization sites (`"_specializations"` and `"_forThis"`) are omitted: a specialized definition keyword expects the same *Type*, characters, and implicit variables as at the property's or function's original definition site.

Site	Expected *Type*, provenance, and access characters (***Any*** & **`Get`** unless otherwise specified)	Implicit variables
Global variable value (**`"instances"`**)	`ValueDomain`	— (other globals resolve globally)
DomainConcept property **`"default"`**	the property's **`"valueDomain"`**	the concept's properties and functions
DomainConcept property **`"confidenceHalfDecayTime"`**	`Duration`	the concept's properties and functions
DomainConcept property **`"constraint"`**	`Variant<TypeValue, Variation<T>, FunctionCompositionRes<Boolean>>` where *T* is the property's **`"valueDomain"`**	an `instance` of the concept and the concept's properties and functions
DomainConcept property **`"hooks"`** (**`"pre"`**/**`"post"`**)	`FunctionComposition`	an `instance` of the concept and the concept's properties and functions, plus the (other) arguments of the hooked *Function*
DomainConcept property **`"computations"`** `JSON array` entry	`FunctionComposition`	an `instance` of the concept and the concept's properties and functions
DomainConcept function **`"default"`** (non-**`static`**)	`CustomFunction`	an `instance` of the concept and the concept's properties and functions
DomainConcept function **`"default"`** (**`static`**)	`CustomFunction`	—
DomainConcept **`"initialization"`**	`FunctionComposition`	an `instance` of the concept and the concept's properties and functions
DomainConcept **`"consolidation"`**	`FunctionComposition`	an `instance` of the concept and the concept's properties and functions
ValueDomain instantiation `JSON Schema` **`"default"`**	the *Type* being defaulted	—
Function **`"_defaultArgumentValues"`**	the argument's declared *Type*	*Function*'s **`interface`** arguments
Function **`"procedure"`**	`FunctionComposition`	*Function*'s **`interface`** arguments
`CustomFunction` `"procedure"`	`FunctionComposition`	`CustomFunction`'s `"interface"` arguments
Function argument value in a *Function* evaluation or `FunctionComposition` expression	the argument's declared *Type*; provenance and access characters from *Function*'s **`"interface"`**	—

9 Definition of Global Variables

Unlike concepts, which are declared under the **"concepts"** key (Sections 4, 5 and 7), the concrete values a Concept Hierarchy program manipulates—its global database of instances and values—live under a separate top-level key, **"instances"**, of the Concept Hierarchy `JSON object`. Its value is a `JSON object` that maps each variable name to an expression defining that variable's value. These bindings are the *global variables* of the language: each is visible, as a variable, in every scope and expression of their Concept Hierarchy.

The expected *Type* of a global variable's value is `ValueDomain`: any expression of any *ValueDomain* is a legal variable value. A variable is therefore always typed by a *ValueDomain*, never by a *DomainConcept* directly. *DomainConcepts* reach a variable only through the reification of section 6: a variable ranging over the instances of a *DomainConcept* C is typed `Instance<`C`>`, and one ranging over a Boolean combination of *DomainConcepts* is typed by a `Variant<...>` of such applications.

Two further conditions govern the well-formedness of **"instances"**. First, a variable may *alias* another, and every alias chain must terminate at a concrete binding. The syntax for defining a variable alias is a `JSON string` that names another variable or another variable alias. Second, a variable's defining expression may reference other variables; this reference relation, resolved by a topological sort over the bindings, must be acyclic.

A binding whose value instantiates an `Instance<...>` carries a name twice: once as the binding's own, and once as the `"instanceName"` of subsection 6.3. These are one name, not two: the `"instanceName"` of a bound instantiation is required to be the binding's name, and may be omitted, in which case the binding's name supplies it. An instance is thus reached, throughout the Concept Hierarchy, exactly as any other variable is — by name, in scope. **"instances"** is where a domain entity acquires such a name, but it is not the only place one may be given, and the two remaining cases differ in when the name comes to exist. An instantiation written elsewhere in the Concept Hierarchy text names its entity by the same `"instanceName"`; an entity so named that no **"instances"** entry binds becomes a binding of that name in its own right, subject to the conditions of definition 9.1 exactly as a written binding is. A Concept Hierarchy program may dynamically create instances during its execution, and each acquires its name in the same way, from the `"instanceName"` of the instantiation that creates it; that the set of bound names grows in this way is a property of a Concept Hierarchy's execution and is outside the scope of this document (subsection 1.2)[25].

Definition 9.1 (Well-formed instances)**.** The **"instances"** section of a Concept Hierarchy is well-formed if and only if (i) every binding's value is an expression of expected type `ValueDomain`; (ii) every alias chain terminates at a concrete binding; (iii) the "defined-in-terms-of" relation induced by inter-variable references is acyclic; (iv) every binding whose value is an `Instance<...>` instantiation specifying an `"instanceName"` gives that argument the binding's own name; and (v) no binding's value is of *Type* `FunctionComposition` or of a subtype of it (subsubsection 7.3.3).

10 External File Inclusion

A Concept Hierarchy of realistic size is seldom written as a single file, and the **"external"** keyword allows knowledge of a single hierarchy to be distributed across several files. This is inclusion in the sense of *customizing* or *extending* an existing body of knowledge, not of *stating a compilation requirement*: it resembles the way a `JSON Schema` references another schema more than the way a `C++` translation unit `#include`s the declarations it needs. It is essential to distinguish this from the merging of two Concept Hierarchies. Merging two hierarchies would require reconciling independently complete definitions that may legitimately share a concept or property name, and hence a conflict-resolution policy. Inclusion performs no such reconciliation: the included files are *fragments* of a single hierarchy rather than hierarchies in their own right, and assembling them yields exactly one **"concepts"**/**"instances"** pair, on which the well-formedness conditions of the preceding sections are then checked. A given file may be reused as a fragment of several distinct hierarchies, but within any one hierarchy, it may be included at most once.

Definition 10.1 (Inclusion sites)**.** External files may be included at exactly three syntactic positions, distinguished by what the referenced file is expected to contain.

[25] The **"instances"** definition declares the instances a Concept Hierarchy starts out knowing, not the instances it may come to know.

1. *Top level.* As a sibling of **"concepts"** and **"instances"**, **"external"** names a fragment file: a JSON object whose **"concepts"** and **"instances"** members are unioned into the corresponding members of the including hierarchy. Such a fragment carries only **"concepts"**, **"instances"**, and possibly a nested **"external"**; a **"name"** or **"metadata"** member is not permitted in an included fragment.

2. *Within a section.* Nested inside **"concepts"** (resp. **"instances"**), **"external"** names a file expected to contain only a JSON object of concept definitions (resp. instance definitions), unioned into that section.

3. *In place of a concept's data.* Where a concept definition would supply a **"data"** object, it may instead give a JSON string interpreted as a path to a file supplying that **"data"**. Only the **"data"** value is externalized; the remainder of the concept definition (**"directParents"**, **"abstract"**, and so on) stays inline. Such a data file may not itself use **"external"**; its content is either the **"data"** JSON object directly or, recursively, a further JSON string path that must ultimately resolve to a **"data"** JSON object.

10.1 Path Resolution

A referenced path is written either as an absolute path or as a path relative to a base directory. A relative path is resolved first against the directory of the root (top-level) hierarchy file; if no file is found there, it is resolved against the directory of the file in which the **"external"** reference occurs. Resolution is therefore deterministic and root-first: where a relative path would name a file under both directories, the file under the root directory is taken.

10.2 Assembly And Well-Formedness

Assembly reduces to a union of definitions governed by a single well-formedness condition. The definitions contributed by the root file and its transitively included files are assembled by taking the union of the keys of their **"concepts"** objects and, separately, the union of the keys of their **"instances"** objects. The governing condition is key uniqueness: no concept name may be contributed by more than one file, and likewise no instance name. Every further consequence of the design follows from this one condition.

Because a file included twice, either at the top-level or within a section, would contribute its keys twice, the at-most-once inclusion is enforced automatically rather than by a separate check. For the same reason, although an included file may itself use **"external"**, the inclusion structure must resolve to a *tree* and not merely to a directed acyclic graph: a file reached along two distinct inclusion paths — the diamond case — contributes its keys twice and will result in an error. Key uniqueness also makes assembly order-independent, since no contribution can overwrite another.

For an externalized **"data"** value (site 3 of definition 10.1), the chain of string indirections must be finite and terminate in a JSON object; a chain that fails to terminate, in particular a cyclic one, is ill-formed.

The price of this design is that the entire hierarchy must be resident before any of it can be understood: since compilation cannot begin until every included definition is available, a hierarchy's memory footprint scales with the total size of its inclusions rather than with that of any single file. And only after resolving the entire hierarchy are the well-formedness rules presented in this document enforced. For example, the *DomainConcept* property and function name uniqueness can only be checked after all *DomainConcepts* are loaded into the Concept Hierarchy.

References

[Costinescu et al.(2025)] Andrei Costinescu, Luis Figueredo, and Darius Burschka. 2025. A Knowledge Modeling Framework for Household Action Recognition and Task Representation: The Concept Hierarchy. *AI, Computer Science and Robotics Technology* 1, 4 (Feb 2025), 1–39. doi:10.5772/acrt.20240053

[Gibson(1979)] James Jerome Gibson. 1979. *The Ecological Approach to Visual Perception.* Houghton, Mifflin and Company, Boston, Massachusetts, United States.

[International(2017)] Ecma International. 2017. ECMA-404 The JSON Data Interchange Standard. https://www.json.org/json-en.html

[McCarthy(1980)] John McCarthy. 1980. Circumscription—A form of non-monotonic reasoning. *Artificial Intelligence* 13, 1 (1980), 27–39. doi:10.1016/0004-3702(80)90011-9 Special Issue on Non-Monotonic Logic.

[Meyer(1992)] Bertrand Meyer. 1992. Applying "Design by Contract". *Computer* 25, 10 (Oct. 1992), 40–51. doi:10.1109/2.161279

[Musen(2015)] Mark A Musen. 2015. The Protégé Project: A Look Back and a Look Forward. *AI matters* 1, 4 (2015), 4–12. doi:10.1145/2757001.2757003

[Noy and McGuinness(2001)] Natalya F. Noy and Deborah L. McGuinness. 2001. *Ontology Development 101: A Guide to Creating Your First Ontology.* Technical Report KSL-01-05. Stanford Knowledge Systems Laboratory. Also Stanford Medical Informatics Report SMI-2001-0880.

[Reiter(1980)] R. Reiter. 1980. A logic for default reasoning. *Artif. Intell.* 13, 1–2 (April 1980), 81–132. doi:10.1016/0004-3702(80)90014-4

[Wright and Andrews(2018)] Austin Wright and Henry Andrews. 2018. *JSON Schema: A Media Type for Describing JSON Documents.* Internet-Draft draft-handrews-json-schema-01. IETF. https://json-schema.org/draft-07/draft-handrews-json-schema-01 Work in progress.

[Wright et al.(2018)] Austin Wright, Henry Andrews, and Geraint Luff. 2018. *JSON Schema Validation: A Vocabulary for Structural Validation of JSON.* Internet-Draft draft-handrews-json-schema-validation-01. IETF. https://json-schema.org/draft-07/draft-handrews-json-schema-validation-01 Work in progress.

A Keyword Summary

Table 13: Summary of Concept Hierarchy keywords. Dots represent the nested level of the keyword relative to a concept's definition.

Keyword	Concept Hierarchy					Notes
		DomainConcept	*ValueDomain*	*Function*	Expression	
concepts	■					Contains the definition of all concepts
instances	■					Contains the initial definition of instances
name	■					Name of the defined Concept Hierarchy
metadata	■					Non-functional Concept Hierarchy data
external	■					For including external files
·**data**		■	■	■		Contains concept-type specific data
·**description**		■	■	■		Optional description/explanation
·**directChildren**		■	■	■		Enumeration of *all* direct child concepts
·**directParents**		■	■	■		Required (except for `Concept`)
·**distinctFrom**		■	■	■		Not instances of distinct group concepts
·**distinctGroup**		■	■	■		Distinct direct child concepts
·**abstract**		■	■	■		Whether the concept is abstract
·**minInstances**		■	■	■		Min. nr. of required concept instances
·**maxInstances**		■	■	■		Max. nr. of required concept instances
··**properties**		■				Property definitions
····**description**		■				Property description/explanation
····**valueDomain**		■				Property type
····**static**		■				Static property marker
····**constraint**		■				Property type restriction
····**hooks**		■				Consistency forward propagation
·····**"pre"**		■				Pre-hook keyword
·····**"post"**		■				Post-hook keyword
····**computations**		■				Consistency backward computation
····**confidenceHalfDecayTime**		■				Property confidence decay rate
····**default**		■				Property OWA/CWA toggle
··**functions**		■				Function definitions
····**description**		■				Function description/explanation
····**default**		■				Default function value
····**static**		■				Static function marker
···**_specializations**		■				Specialize property and function data
····**_forThis**		■				Specialize data for this concept only
··**management**		■				Ensuring consistency of instances
···**initialization**		■				Instance consistent after creation
···**consolidation**		■				Consistent after creation & cloning
··**templateContext**			■	■		Definition of template parameters
···**order**			■	■		Template parameter order
···**substitution**			■	■		Template substitution
···**variadicGroupIdentifiers**			■	■		Maps arguments to variadic groups
··**instantiation**			■			Defines `JSON Schema` of literal `JSON` value
··**defaultSerialization**			■			Infers expression *Type* when undefined
··**interface**				■		Function signature
···**_defaultArgumentValues**				■		Default argument values
···**res**				■		Return value
··**procedure**				■		Function body
··**addNewVariablesInExistingScope**				■		Adds new variables in scope
··**subScopes**				■		Defines scopes for arguments
·····***Any***			■	■		Any value provenance allowed
·····***Addr***			■	■		Only addressable values allowed
·····***ResetAddr***				■		As above, but value is reset before use
·····**Get**				■		Only read value
·····**Modify**				■		Overwrite (no read) value
·····**GetModify**				■		Read and write value
isFunctionEvaluation					■	Discriminate composition from evaluation

B Features Enabled By *ValueDomain* Definitions

Table 14: Features enabled in the Concept Hierarchy by defining the respective *ValueDomains* and *Functions*.

ValueDomains	**Feature enabled**
`ValueDomain`	defining *DomainConcept* properties and functions
`InstanceBase`	defining *DomainConcept* properties relating other *DomainConcepts*
`Instance<AcceptConcepts..., RejectConcepts...>`	restricting instances by concept affordance
`Variant<T...>`	union of *Types* and defining the disjunction of *DomainConcepts*
`Variation<T>`	for defining subsets of a *Type* and implicit property **`constraints`**
`VariationConjunction<T>`	for defining the intersection of subsets
`VariationDisjunction<T>`	for defining the union of subsets
`VariationNegation<T>`	for defining the complement of a subset
`SubTypeVariation<T, SubT>`	defining a property's **`constraint`** as a `TypeValue`: subtype of **`valueDomain`**
`InstanceDataBase`	used in the **`instantiation`** schema of `InstanceBase`
`InstanceData<AcceptConcepts...>`	used in the **`instantiation`** schema of any `Instance<AcceptConcepts..., RejectConcepts...>`
`ConceptValue`	used in the **`instantiation`** schema of `InstanceBase` and `Instance`
`CustomFunction`	defining *DomainConcept* functions
`FunctionComposition`	defining the *`procedure`* of *CustomFunctions*
`FunctionCompositionRes<T>`	ensuring that the composition returns a value of *Type* *`T`*
`TypeValue`	used in the instantiation schema of `InstanceBase` and `Instance`
`Boolean`	defining an explicit **`constraint`** check *Function*
`Duration`	defining **`confidenceHalfDecayTime`** values
`Optional<T>`	defining properties that may not have a value ($\not\equiv$ an unknown value)
Functions	**Feature enabled**
`Function`	defining modifications of *ValueDomain* values
`Assign<T>`	setting values
`AssignMeasured<T>`	setting a value with its confidence and setting the confidence of properties' 0th **`computations`** derivation
`CreateEmptyVariation<T>`	defines a `Variation<T>` representing the empty set of values of *Type* *`T`*
`CreateAnyVariation<T>`	defines a `Variation<T>` accepting the whole set of values of *Type* *`T`*
`Evaluate`	evaluating a `FunctionComposition` that must not return any value
`EvaluateRes<T>`	evaluating a `FunctionCompositionRes<T>` that must return a *`T`*-value
`EvaluateResAddr<T>`	evaluating a `FunctionCompositionRes<T>` that must return an ***`Addr`*** *`T`*-value
`EvaluateFunction`	evaluating a `CustomFunction` that must not return any value or evaluating a `Function` instance that does not have **`res`** defined
`EvaluateFunctionRes<T>`	evaluating a `CustomFunction` that must return a *`T`*-value or evaluating a `Function` instance that has **`res`** defined of a subtype of *`T`*
`EvaluateFunctionResAddr<T>`	evaluating a `CustomFunction` that must return an ***`Addr`*** *`T`*-value or evaluating a `Function` instance that has **`res`** defined of a subtype of *`T`* with ***`Addr`*** provenance
`Continue`	defining the signal to skip to the next iteration of a loop-like *Function*
`Break`	defining the signal to stop the loop-like *Function*
`Stop`	defining the signal to stop a *Function* evaluation without returning a value
`Return<T>`	defining the signal to stop a *Function* evaluation and return a *`T`*-value
`WithConfidenceWitness<T>`	changing the confidence of an argument's value by the confidence of a *`witness`* argument, which does not affect the value of the argument